

UNIVERSIDAD NACIONAL DE MAR DEL PLATA

Facultad de Ingeniería

Trabajo final

**Desarrollo de un Analizador Vectorial de Redes para
medición no invasiva de glucemia**

Victoria Carla Torres



Ingeniería Electrónica
Departamento de Ingeniería Electrónica y Computación
Septiembre 2026

Agradecimientos

Con todo mi cariño, quisiera agradecer:

En primer lugar, a mi director Ramiro Avalos Ribas. Por ser mi mentor y referente, además de tutor informal y apoyo moral estos cuatro años que he tenido el placer de compartir el espacio académico. Su paciencia, consejos, y forma de resolver los problemas son los valores más importantes que me llevo de la carrera.

Con mucho respeto, a mis directores de beca de investigación Alejandro José Uriz y Jorge Castiñeira Moreira. Por su constante disposición, por compartir mis alegrías y mis indignaciones, por su trabajo para darme las herramientas que facilitaron mi camino.

A mi codirector Brian Maximiliano Gluzman, por compartir mi terquedad en momentos donde hubo que resolver un problema. Por enseñarme a soldar.

A Esteban Lucio González, quien me introdujo al Laboratorio y a la investigación, por confiar en mi potencial. Espero poder retribuir esa confianza dando todo de mí para ser una profesional íntegra y comprometida.

A mis compañeros del Laboratorio de Comunicaciones, que nunca escatiman en brindar su apoyo sin importar el tipo de problema con el que me acerque, académico o extraacadémico. Gracias a ellos encontré un espacio de aprendizaje y crecimiento para dar mis primeros pasos en el ámbito de la investigación, pero sobre todo, un lugar de pertenencia. Entre ellos, quiero incluir a Leonardo David Vazquez, cuyo trabajo final sentó las bases para el desarrollo de este. Su emoción por el proyecto fue contagiosa, y estuvo presente (aún estando lejos) respondiendo dudas, llevando seguimiento y dando aliento en todo momento.

A Gustavo Arenas, por permitirme pasar la mayor parte de mi trayectoria como estudiante siendo ayudante en la asignatura Física 2/B. La docencia es una parte muy importante de quien soy, y estoy muy agradecida de ser y haber sido parte de una de las cátedras más lindas de mi recorrido como estudiante.

A mis compañeros y docentes. Los temas pesados se hacen llevaderos cuando el ambiente de estudio es bueno. Porque tenemos la gran suerte, con casi todos los docentes en electrónica, de poder tocarles la puerta y que nos reciban para despejar cualquier duda, aunque no competa su materia.

A mis amigos, porque estoy segura de que sin ellos no es posible sobrellevar el nivel de exigencia que demanda una carrera en ingeniería. Por las largas horas de estudio, rondas de mates, charlas interminables soñando con el día en que nos recibamos, juegos de truco en el comedor entre clase y clase, "amiga te juro que vos podés", apoyo en momentos de felicidad y otros de mucho estrés y dificultades, personales y académicas. Algunas de estas personas se han convertido en mi familia, y es imposible imaginar mi vida sin ellas.

A mi familia, gracias por soportar mis momentos de ansiedad y estrés, por su apoyo constante y fé ciega en mí.

Finalmente, expreso mi reconocimiento a la Universidad Nacional de Mar del Plata y en especial a la Facultad de Ingeniería. A lo largo de estos años se ha convertido en más que un espacio de formación, en un lugar de pertenencia del cual me enorgullezco enormemente. Sin instituciones públicas de calidad, muchos no tendríamos hoy la oportunidad de estudiar una carrera universitaria.

Victoria C. Torres

5/9/2026

Índice

Agradecimientos	1
Siglas y abreviaturas	4
Resumen	4
1 Introducción	6
1.1 Contexto y motivación	6
1.2 Partes interesadas	7
1.3 Objetivos	7
1.4 Organización del informe	7
2 Marco teórico	8
2.1 Analizador Vectorial de Redes	8
2.2 Parámetros distribuidos (S)	8
2.3 Calibración	9
2.3.1 Errores	9
2.3.2 Método SOL	9
2.4 Acoplador Direccional	10
2.4.1 Puente resistivo	10
2.5 Reglas de diseño en RF	11
2.6 Heterodinación de señales	11
2.7 Procesamiento digital de señales	12
2.7.1 Filtros digitales	12
2.7.2 Transformada Rápida de Fourier	14
2.7.3 Estimación del S11	14
3 Proyecto	16
3.1 Descripción general del producto	16
3.2 Hardware	16
3.3 Firmware	17
3.4 Software	17
3.5 Interfaz de usuario	17
4 Resultados	19
5 Conclusiones y trabajos futuros	20
6 Esquemáticos y código	21
6.1 Esquemáticos Altium	21
6.1.1 Generador de RF incidente (ADF4351)	22
6.1.2 Generador de RF Oscilador Local (ADF4351)	23

6.1.3	Mezclador	24
6.1.4	Alimentación y CLK	25
6.1.5	Acoplador Direccional	26
6.1.6	Amplificadores de FI	27
6.1.7	Atenuadores	28
6.2	Interfaz de usuario	29
6.2.1	Core	29
6.2.2	Graphical User Interface	36
Apéndices		49
A	Estudio Preliminar	51
B	Plan de Trabajo	55
C	Especificación de Requerimientos	59
D	Especificación Funcional	65
E	Especificación Técnica	73
F	Plan de Pruebas	85
Bibliografía		91

Siglas y abreviaturas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
UHF	Ultra High Frequency
LAC	Laboratorio de Comunicaciones
ICYTE	Instituto de Investigaciones Científicas y Tecnológicas en Electrónica
FI	Frecuencia Intermedia
DUT	Device Under Test
RF	Radiofrecuencia
PCB	Printed Circuit Board
FFT	Fast Fourier Transform
IA	Inteligencia Artificial

Resumen

El proyecto propone el diseño, implementación, calibración y ensayo de un Analizador Vectorial de Redes (VNA) de un puerto, que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 hasta los 2,2 GHz, adaptado para operar en conjunto con un sensor de glucemia no invasivo.

El principio de medición se basa en la existencia de una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [14], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

Capítulo 1

Introducción

1.1 Contexto y motivación

La diabetes es una afección crónica que se desencadena cuando el organismo pierde su capacidad de producir suficiente insulina o de utilizarla con eficacia para mantener los niveles de glucosa en la sangre dentro de los márgenes normales. La cantidad de adultos que la padecen se ha casi cuadruplicado entre 1980 y 2016, con un aumento en la prevalencia más acelerado en países de ingresos bajos y medios que en países de ingresos altos [11]. Para 2021 fue la causa directa de 1,6 millones de muertes [10]. Existen numerosos dispositivos que permiten realizar un control de la glucosa en sangre pero requieren, en general, de una punción en alguna parte del cuerpo para extraer una muestra de sangre. El mecanismo tradicional requiere depositar la muestra en una tira reactiva dentro de un medidor para obtener el valor de glucemia. Se recomienda, en promedio, que los pacientes realicen 4 mediciones al día (dependiendo de la estabilidad de los niveles o del régimen de tratamiento). Dado que la extracción suele ser dolorosa, además de que tiene un costo asociado al gasto de cada tira reactiva, la mayoría de los pacientes no logran tener un buen monitoreo de su glucemia. Por este motivo se ha incursionado en la glucometría semi-invasiva como los sistemas CGM (Continuous Glucose Monitoring) [1] que obtienen mediciones del líquido intersticial y requieren la colocación de un sensor debajo de la piel que se debe reemplazar periódicamente, por lo que siguen siendo invasivos, incómodos de usar y descartables. El estudio de propuestas para la medición no invasiva ha avanzado en los últimos años, pero aún no existe una técnica ampliamente adoptada ni dispositivos comerciales fiables. Se ha observado que existe una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [14], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

Por esta razón el Laboratorio de Comunicaciones (LAC), perteneciente al Instituto de Investigaciones Científicas y Tecnológicas en Electrónica (ICYTE), que depende de la UNMDP y del Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET) tiene una línea de investigación cuyo objetivo es diseñar e implementar un prototipo que permita realizar una medición de glucemia, basado en un VNA y un sensor de microondas.

Un Analizador Vectorial de Redes es un instrumento de medición que permite caracterizar sistemas en alta frecuencia. La forma en que lo hace es haciendo incidir una señal de microondas en los puertos del sistema, y midiendo las señales que salen del resto de los puertos.

El Laboratorio busca optimizar, compactar y mejorar el rendimiento del VNA actual, por lo que requiere el diseño y desarrollo de un VNA de un puerto que esté adaptado a un sensor de microondas que resuena alrededor de los 2 GHz.

1.2 Partes interesadas

El Grupo de Investigación del LAC es quien solicita el desarrollo. Por esta razón, el equipo se utilizará bajo su supervisión, y sus posibles usuarios son:

- Ingenieros, técnicos y/o radioaficionados;
- Estudiantes de ingeniería.

1.3 Objetivos

El objetivo principal del proyecto es el diseño, desarrollo, calibración y ensayo de un VNA de bajo costo y tamaño compacto, que opere en la banda de frecuencias de UHF que abarca desde 1,8 a 2,2 GHz, orientado a la medición no invasiva de glucemia.

Los objetivos particulares son:

- Implementar en una placa de circuito impreso el VNA;
- Generar un barrido en la banda de frecuencias de interés;
- Implementar un sistema heterodino que permita trabajar con señales de frecuencia intermedia (FI) en la banda de audio;
- Digitalizar las señales obtenidas para aplicar un tratamiento digital que permita calcular el coeficiente de reflexión del sistema bajo prueba (DUT - *Device Under Test*);
- Implementar una rutina de calibración para eliminar errores sistemáticos del sistema;

1.4 Organización del informe

El informe se organiza de la siguiente manera: en el capítulo primero se introduce el proyecto y se brinda un punteo de los objetivos más importantes. El capítulo 2 se centra en el marco teórico, donde se explican las bases para entender el funcionamiento del dispositivo. El capítulo 3 describe el proyecto con su implementación. En el capítulo 4 se muestran los resultados y mediciones. El capítulo 5 se dedica a conclusiones y trabajos futuros, y el 6 exhibe los esquemáticos y códigos utilizados. Por último se anexan los apéndices en donde se encuentran: el estudio preliminar, el plan de trabajo, las especificaciones de requerimientos, funcional y técnica y el plan de pruebas.

Capítulo 2

Marco teórico

2.1 Analizador Vectorial de Redes

El Analizador Vectorial de Redes fue definido en sus inicios como un instrumento que permite medir los parámetros de redes eléctricas [6]. En su esquema más básico posee una fuente de radiofrecuencias (RF), que genera una señal de estímulo para el dispositivo bajo prueba, y un banco de receptores que miden las señales que salen del DUT y las comparan con la onda incidente. Los resultados se procesan y muestran al usuario por medio de alguna interfaz.

Dependiendo de los dispositivos que se buscan caracterizar, se determina la cantidad de puertos que debe tener el VNA. Un puerto permite medir el parámetro de reflexión. Con dos puertos, se puede medir además el de transmisión. Si posee un solo canal, se puede medir la transmisión inversa y la reflexión en la salida invirtiendo físicamente el dispositivo bajo prueba. Este paso se puede evitar agregando un canal, lo que incrementa el costo del VNA.

Impedancia de entrada/S11

2.2 Parámetros distribuidos (S)

Los parámetros distribuidos, también llamados S por su nombre en inglés (*Scattering*) permiten caracterizar redes de dos o más puertos en función de sus ondas entrantes y salientes. Se especifican para una impedancia de referencia y a una frecuencia específica, y son útiles en altas frecuencias porque otros tipos de parámetros (como los Z o los Y) no pueden ser medidos directamente.

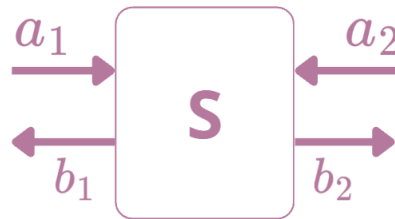


Figura 2.1: Parámetros S

Según las ondas incidentes a_1 y a_2 y las reflejadas b_1 y b_2 , los parámetros S se definen como:

$$S_{11} = \left. \frac{b_1}{a_1} \right|_{a_2=0}; S_{12} = \left. \frac{b_1}{a_2} \right|_{a_1=0}; S_{21} = \left. \frac{b_2}{a_1} \right|_{a_2=0}; S_{22} = \left. \frac{b_2}{a_2} \right|_{a_1=0}$$

Para lograr que las a sean iguales a 0, el puerto al que pertenecen debe estar adaptado, garantizando que no haya ninguna señal incidente. El parámetro S_{11} se conoce como el coeficiente de reflexión en el puerto 1 cuando el puerto 2 está adaptado.

2.3 Calibración

2.3.1 Errores

Los errores que pueden aparecer en una medición se pueden dividir en dos grandes clases: los errores sistemáticos y los aleatorios. Los primeros se pueden corregir, mientras que los segundos se encuentran en toda medición y están fuera del control del usuario.

En el caso de dispositivos electrónicos algunos errores sistemáticos son los producidos por variaciones en la potencia de salida de los generadores, pérdidas de inserción en materiales conductores y conectores, desfasajes ocasionados por diferencias en longitudes eléctricas o retardos en los circuitos integrados, reflexiones generadas por desadaptación de impedancias y deriva térmica. Algunos errores aleatorios pueden ser ruido térmico, de cuantización e interferencia electromagnética.

La calibración en un VNA permite corregir los errores sistemáticos, excluyendo el ruido por deriva térmica. Este se puede corregir solo si la calibración se realiza inmediatamente antes de la medición.

2.3.2 Método SOL

En VNAs de un puerto se pueden simplificar los errores a un modelo de tres términos que describen la directividad, adaptación del puerto y errores de seguimiento (variaciones de amplitud y fase en función de la frecuencia)[12]. Usando tres cargas distintas de valores conocidos, se puede formar un sistema de tres ecuaciones con tres incógnitas y resolver para obtener los términos del error:

$$e_{00} + \Gamma_1 \Gamma_{M1} e_{11} - \Gamma_1 \Delta_e = \Gamma_{M1} \quad (2.3.1a)$$

$$e_{00} + \Gamma_2 \Gamma_{M2} e_{11} - \Gamma_2 \Delta_e = \Gamma_{M2} \quad (2.3.1b)$$

$$e_{00} + \Gamma_3 \Gamma_{M3} e_{11} - \Gamma_3 \Delta_e = \Gamma_{M3} \quad (2.3.1c)$$

Donde Γ_M es el parámetro de reflexión medido para cada carga, y $\Delta_e = e_{00}e_{11} - (e_{10}e_{01})$. Los términos del error se pueden observar en la figura 2.2.

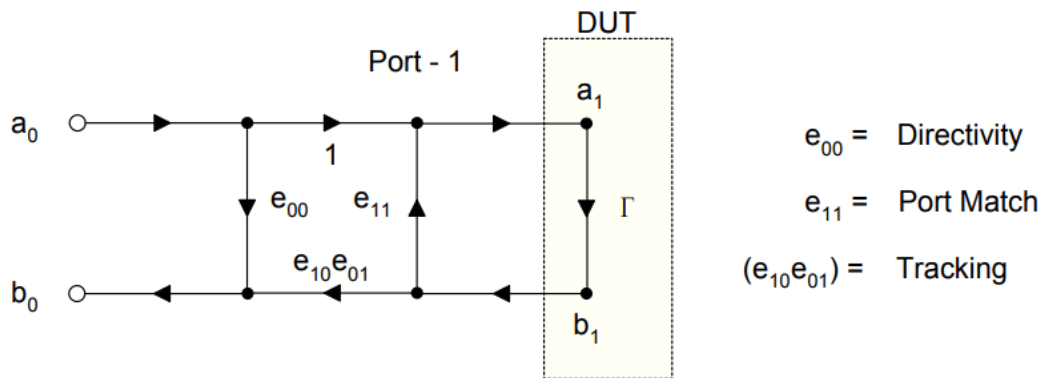


Figura 2.2: Términos de error para un VNA de un puerto

Si las cargas independientes son un circuito abierto, cortocircuito y carga de 50Ω entonces el método de calibración se denomina SOL (Short, Open, Load) y las ecuaciones (2.3.1) se simplifican a:

$$e_{00} - \Gamma_{MS}e_{11} + \Delta_e = \Gamma_{MS} \quad (2.3.2a)$$

$$e_{00} + \Gamma_{MO}e_{11} - \Delta_e = \Gamma_{MO} \quad (2.3.2b)$$

$$e_{00} = \Gamma_{ML} \quad (2.3.2c)$$

Luego el Γ real se calcula como:

$$\Gamma = \frac{\Gamma_M - e_{00}}{\Gamma_M e_{11} - \Delta_e} \quad (2.3.3)$$

2.4 Acoplador Direccional

En la mayoría de diseños en electrónica, el acoplamiento electromagnético entre líneas de transmisión se considera ruido porque interfiere con la señal que se busca transmitir. Una excepción es el caso de acopladores, donde el acoplamiento es un parámetro de diseño que permite medir la señal de una línea con un alto aislamiento, es decir, sin interrumpir su paso. Además cuando se dispone de un par de líneas acopladas, se puede verificar que en uno de sus puertos la señal incidente se anula, generando un punto que permite medir la onda reflejada.

Las principales propiedades que caracterizan un acoplador son la directividad, el aislamiento, el acoplamiento y la pérdida de inserción [13]. La directividad mide la capacidad de separar las ondas que se propagan en distintas direcciones. El aislamiento, la potencia que llega a la carga desacoplada. El acoplamiento indica la potencia que se entrega al puerto acoplado. La pérdida de inserción tiene en cuenta cuánta potencia que se debería transmitir se pierde por ser entregada a los puertos acoplados y aislados, y tiene en cuenta las pérdidas en los materiales (dieléctrico y resistividad de las pistas).

Un acoplador ideal es un dispositivo pasivo, que posee directividad y aislamiento infinitos.

Existen acopladores direccionales como el HP 778d [7] que posee una directividad mayor a 36dB y pérdidas por inserción menores a 0,6dB, pero tiene un ancho de banda que permite hacer mediciones entre 100 MHz y 2 GHz, y sus dimensiones físicas y peso hacen que su utilidad se limite a mediciones en laboratorio. Para soluciones integradas hay alternativas comerciales como el ZABDC20-322 de Minicircuits [9] pero su costo hace que el diseño no sea escalable.

2.4.1 Puente resistivo

Una solución compacta, económica y con un ancho de banda teórico infinito es el puente resistivo tipo Wheatstone. Como se puede observar en la figura 2.3 es una red simple de comparación de impedancias, formada por cuatro resistencias sobre las cuales se inyecta una señal de referencia en la parte superior. Una de las resistencias es variable (Z_x), mientras que el resto son iguales entre sí. Se utiliza un voltímetro para medir la diferencia de potencial entre las ramas. Si la resistencia variable es de 50Ω , el puente queda balanceado y el voltímetro indica un cero. Si es menor a 50Ω , la rama izquierda presenta un voltaje menor a la derecha, mientras que para valores mayores ocurre lo opuesto. Si la señal incidente es sinusoidal es posible además caracterizar cargas reactivas midiendo el desfase de la señal Z_x respecto de la incidente. Este es el principio que permite al puente comportarse como un acoplador direccional: la diferencia de potencial entre las ramas es función de la onda reflejada equivalente.

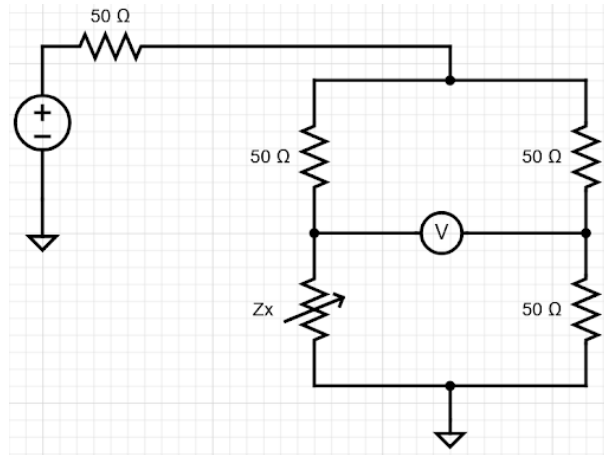


Figura 2.3: Esquema básico de un puente de Wheatstone

2.5 Reglas de diseño en RF

Al momento de rutear un PBC (*Printed Circuit Board* - Placa de Circuito Impreso) se deben tener en consideración algunas buenas prácticas que ayudan a mantener la integridad de las señales, la impedancia de las líneas y evitar la interferencia electromagnética, sobre todo cuando se trabaja en RF.

Se debe garantizar que las pistas cumplan con la impedancia requerida de 50Ω . Esto implica calcular el ancho de la línea usando las ecuaciones de diseño según el tipo de línea o se pueden encontrar calculadoras online o integradas en los software de diseño con este fin. Se deben evitar ángulos rectos, que causan discontinuidades y aumentan la capacitancia parásita.

El plano de masa que se encuentra debajo de las pistas de RF debe ser continuo, para que la corriente de retorno pueda seguir el camino de menor inductancia. Por esta razón no se recomienda rutear en el plano inferior al de las rutas de RF, pero si se necesita hacerlo no se deben atravesar las regiones donde en la capa superior haya trazos de RF.

Se deben usar las técnicas *vía stitching* y *vía shielding*. La segunda se usa para reducir la interferencia electromagnética y el crosstalk [3], mientras que la primera "ata" los planos de masa de distintas capas, ayudando a mantener baja impedancia y caminos de retorno cortos. Además evita resonancias dentro de la placa.

Los capacitores de desacople se deben ubicar lo más cerca posible de los integrados, de forma de poder suministrar rápidamente los picos de corriente que demandan los integrados que la fuente de alimentación no puede entregar [2]. También deben utilizarse cuando se suministra alimentación a un trazo a través de vías.

Los trazos deben ser lo más cortos posible, para minimizar la atenuación de las señales y reflexiones indeseadas.

2.6 Heterodinación de señales

Las señales que se intentan medir en un VNA son de naturaleza analógica, por lo que para realizar el cálculo del parámetro de reflexión o S11 primero se debe poder medir su amplitud y fase. Existen integrados que pueden obtener la magnitud de una onda de RF como el HMC601 [5] pero se pierde información de la fase. Por eso se busca una forma de bajar la frecuencia de la señal para poder adquirirla de forma digital.

Un esquema heterodino se basa en la combinación de una señal de RF de interés con otra señal generada localmente por un oscilador local (OL o LO por sus siglas en inglés) desplazada en frecuencia un valor constante, llamado frecuencia intermedia (FI). Es posible demostrar matemáti-

camente que cuando se multiplican idealmente en el dominio del tiempo dos señales sinusoidales, a la salida existen componentes espectrales en la suma y diferencia de las frecuencias originales. Mediante un filtrado se puede aislar la componente diferencia, logrando así trasladar la información de la señal de RF hacia la FI. Utilizando un mezclador doble se pueden heterodinar dos señales con un solo oscilador local, como muestra la figura 2.4 . De esta forma obtenienen las señales incidente y reflejada, conservando su amplitud y fase, en una frecuencia de audio que facilite su adquisición y procesamiento.

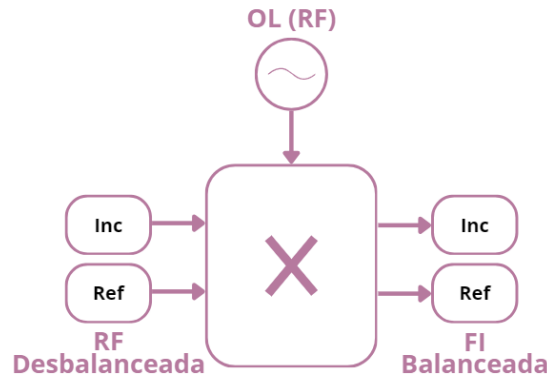


Figura 2.4: heterodinación

2.7 Procesamiento digital de señales

Para poder aplicar un procesamiento digital cualquier señal analógica continua en el tiempo debe ser muestreada, es decir convertirla en una secuencia de números de precisión finita. El proceso implica una pérdida de información de los intervalos intermedios, pero según el teorema de Nyquist-Shannon una señal de banda limitada puede reconstruirse inequívocamente a partir de sus muestras si la frecuencia de muestreo f_s cumple la condición $f_s \geq 2f_{\max}$, donde $f_{\max}$ representa la componente frecuencial de mayor valor de la señal [8].

Una vez que la señal está digitalizada, para poder extraer su amplitud y fase, sabiendo que se encuentra a una frecuencia de valor FI, se debe aplicar un filtrado digital y realizar la Transformada Rápida de Fourier (FFT - *Fast Fourier Transform*)

2.7.1 Filtros digitales

El objetivo principal de la etapa de filtrado es aislar la señal de FI eliminando el ruido de banda ancha, las interferencias de baja frecuencia y armónicos de orden superior.

La señal muestreada $x[n]$ pasa por un filtro pasabanda digital caracterizado por una respuesta al impulso $h[n]$. La operación se define mediante la convolución discreta:

$$y[n] = x[n] * h[n] = \sum_{k=0}^M h[k] x[n-k] \quad (2.7.1)$$

donde M representa el orden del filtro. Los coeficientes $h[k]$ se diseñan para configurar una banda de paso en torno a la frecuencia de FI.

Los filtros digitales se pueden clasificar en dos grandes familias según la naturaleza de su respuesta al impulso: filtros de respuesta finita (*Finite Impulse Response*, FIR) y filtros de respuesta infinita (*Infinite Impulse Response*, IIR).

Un filtro IIR se caracteriza por una respuesta al impulso de duración infinita, que se logra mediante una estructura recursiva con realimentación de las salidas anteriores. Su ecuación en

diferencias se expresa como:

$$y[n] = \sum_{k=0}^M b_k x[n-k] + \sum_{k=1}^N a_k y[n-k] \quad (2.7.2)$$

La presencia de los coeficientes a_k introduce polos en el plano z fuera del origen. Si bien permiten lograr bandas de transición abruptas con órdenes bajos, presentan dos desventajas fundamentales: no garantizan fase lineal en toda la banda de paso y pueden ser inestables si sus polos migran fuera del círculo unitario debido a cuantización de precisión finita.

Por otro lado, un filtro FIR presenta una respuesta al impulso de duración finita N_{taps} (comúnmente denominada número de coeficientes o *taps*), siendo $h[n] = 0$ para $n < 0$ y $n \geq N_{\text{taps}}$. Su ecuación en diferencias es puramente no recursiva:

$$y[n] = \sum_{k=0}^M b_k x[n-k] = \sum_{k=0}^{N_{\text{taps}}-1} h[k] x[n-k] \quad (2.7.3)$$

donde $M = N_{\text{taps}} - 1$ representa el orden del filtro. Su transferencia $H(z)$ es causal, estable y realizable. Todos sus polos se ubican de manera trivial en el origen ($z = 0$), lo que garantiza estabilidad incondicional.

Si la secuencia de coeficientes $h[n]$ cumple la condición de simetría par alrededor de su centro ($h[n] = h[N_{\text{taps}} - 1 - n]$), la respuesta en frecuencia presenta la forma:

$$H(e^{j\omega}) = |H(e^{j\omega})| e^{-j\omega\tau_g} \quad (2.7.4)$$

donde el retardo de grupo τ_g es estrictamente constante para todas las frecuencias:

$$\tau_g = \frac{N_{\text{taps}} - 1}{2} \quad [\text{muestras}] \quad (2.7.5)$$

Un retardo de grupo constante asegura que el filtro no introduce distorsión de fase no lineal, preservando la diferencia de fase relativa entre el canal de referencia (V_{ref}) y el de medición (V_{med}).

Diseño de filtros FIR mediante el método del enventanado

El diseño del filtro FIR se fundamenta en el método del enventanado (*window design method*). Este método consiste en truncar y desplazar la respuesta al impulso ideal de duración infinita, $h_d[n]$ (correspondiente a un filtro pasabanda ideal centrado en f_{FI}), multiplicándola punto a punto por una función ventana finita $w[n]$ de longitud N_{taps} :

$$h[n] = h_d[n] \cdot w[n], \quad 0 \leq n \leq N_{\text{taps}} - 1 \quad (2.7.6)$$

Para la síntesis de la respuesta pasabanda se emplea la ventana de Hamming, definida analíticamente como:

$$w[n] = 0,54 - 0,46 \cos\left(\frac{2\pi n}{N_{\text{taps}} - 1}\right), \quad 0 \leq n \leq N_{\text{taps}} - 1 \quad (2.7.7)$$

La multiplicación en el dominio del tiempo equivale a la convolución periódica de la respuesta en frecuencia ideal con el espectro de la ventana, $W(e^{j\omega})$. La ventana de Hamming ofrece un compromiso óptimo para el filtrado de Frecuencia Intermedia:

- ****Atenuación en banda de rechazo:**** Proporciona un nivel de lóbulo secundario pico de aproximadamente -43 dB y una atenuación mínima en la banda de rechazo de -53 dB, atenuando eficazmente interferencias fuera de banda y el ruido térmico.
- ****Suavizado del fenómeno de Gibbs:**** Elimina las oscilaciones de sobrepico en los bordes de la banda de paso típicas del truncamiento rectangular estricto.

Significado e impacto del parámetro N_{taps}

El parámetro N_{taps} (*numtaps*) define la longitud temporal de la respuesta impulsional del filtro y rige dos compromisos teóricos fundamentales:

1. ****Ancho de la banda de transición (Δf):**** El ancho del área de transición entre la banda de paso y la de rechazo es inversamente proporcional al número de coeficientes. Para la ventana de Hamming, esta relación viene dada aproximadamente por:

$$\Delta f \approx \frac{3,3 \cdot f_s}{N_{\text{taps}}} \quad (2.7.8)$$

donde f_s es la frecuencia de muestreo. Un incremento en N_{taps} estrecha la banda de transición, aumentando la selectividad frecuencial del filtro en torno al tono de 1 kHz.

2. ****Duración del régimen transitorio:**** Debido a la memoria finita del filtro, la respuesta temporal tarda exactamente $N_{\text{taps}} - 1$ muestras en llenar por completo la línea de retardo. Durante este intervalo, la salida se encuentra en régimen transitorio. Por ende, conocer N_{taps} establece matemáticamente el número exacto de muestras iniciales que deben ser descartadas en el algoritmo antes de ejecutar la FFT sobre el régimen estacionario.

2.7.2 Transformada Rápida de Fourier

Para determinar la amplitud y fase de las señales de FI, las secuencias filtradas en el dominio del tiempo se transforman al dominio de la frecuencia mediante la Transformada Discreta de Fourier (DFT), calculada a través del algoritmo de la Transformada Rápida de Fourier (FFT). Para una secuencia de N muestras, la expresión de análisis viene dada por:

$$X[k] = \sum_{n=0}^{N-1} x[n] w[n] e^{-j \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N-1 \quad (2.7.9)$$

donde $w[n]$ representa la función de la ventana aplicada a la secuencia temporal.

Dado que la captura se realiza sobre una ventana de observación finita, si la duración del intervalo de adquisición no contiene un número entero exacto de ciclos de la señal, se produce el fenómeno de fuga espectral (*spectral leakage*). Este fenómeno dispersa la energía de la frecuencia fundamental hacia los bins frecuenciales adyacentes, degradando la precisión en la estimación de amplitud y fase. Para mitigar este efecto, se aplica una función de enventanado $w[n]$. La ventana *Flatop* se caracteriza por presentar un lóbulo principal ancho y plano; esta propiedad minimiza el error de estimación de amplitud, mejorando la exactitud en la medición de niveles de tensión.

2.7.3 Estimación del S11

Una vez computadas las transformadas espectrales complejas del canal de referencia, $X_{\text{ref}}[k]$, y del canal de medición, $X_{\text{med}}[k]$, el algoritmo procede a la identificación sincrónica del tono heterodino de 1 kHz.

En primer lugar, se evalúa el espectro de magnitud del canal de referencia dentro de un rango frecuencial acotado alrededor de f_{FI} (banda de búsqueda del pico) para determinar el índice del bin que concentra la máxima energía:

$$k_{\text{pico}} = \arg \max_{k \in \text{Banda}} |X_{\text{ref}}[k]| \quad (2.7.10)$$

Obtenido el bin característico k_{pico} , se evalúan los valores complejos de ambas transformadas exactamente en dicha posición espectral, obteniendo los fasores complejos $V_{\text{ref}} = X_{\text{ref}}[k_{\text{pico}}]$ y

$V_{\text{med}} = X_{\text{med}}[k_{\text{pico}}]$. El coeficiente de reflexión complejo crudo (no calibrado), $S_{11,\text{raw}}$, para la frecuencia de RF evaluada en el paso del barrido se calcula como el cociente vectorial complejo:

$$S_{11,\text{raw}} = \frac{V_{\text{med}}[k_{\text{pico}}]}{V_{\text{ref}}[k_{\text{pico}}]} \quad (2.7.11)$$

A partir del número complejo $S_{11,\text{raw}}$, se extraen directamente el módulo en escala logarítmica y la fase relativa:

$$|S_{11,\text{raw}}|_{\text{dB}} = 20 \log_{10} |S_{11,\text{raw}}| \quad (2.7.12)$$

$$\phi_{S_{11,\text{raw}}} = \arg(S_{11,\text{raw}}) \quad (2.7.13)$$

Este valor complejo no calibrado sirve como la entrada directa para el posterior algoritmo de corrección de errores vectoriales mediante el modelo de tres términos (SOL).

Proyecto

3.1 Descripción general del producto

Como describe la sección 2 de la Especificación de Requerimientos C el sistema debe generar un barrido de frecuencias en el rango de 1,8 a 2,2 GHz. La señal se debe inyectar en el DUT y se debe medir la onda reflejada. Ambas ondas (incidente y reflejada) deben bajarse a una frecuencia intermedia para luego adquirirlas y digitalizarlas con el fin de realizar el procesamiento correspondiente. Un diagrama en bloques detallado del sistema se puede observar en la figura 3.1. Los datos a asumir, requerimientos y pautas de diseño se pueden encontrar en la Especificación de Requerimientos C.

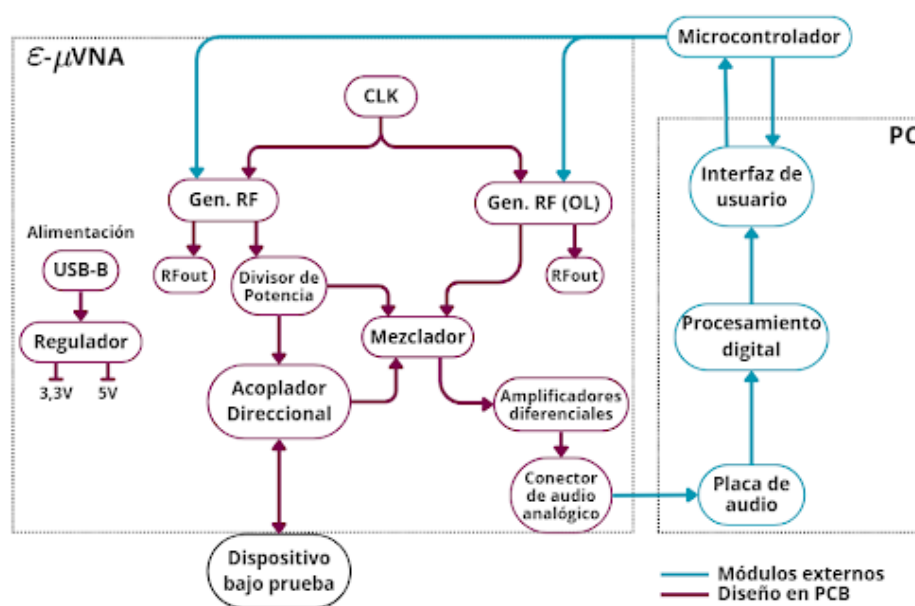


Figura 3.1: Diagrama en bloques del sistema

A continuación, se detalla la solución del hardware, software, firmware e interfaz de usuario.

3.2 Hardware

El hardware se diseñó en su totalidad en el software Altium Designer, con una licencia estudiantil gracias al programa Altium Student Lab [4]. Se creó una librería con los esquemáticos, footprints y modelos 3D de los componentes a utilizar. Se dividió el proyecto en esquemáticos que representan los distintos bloques funcionales. En principio se planteó un modelo de 2 capas,

con FR4 como dieléctrico de 0,8 mm de espesor.

3.3 Firmware

El firmware se programó en lenguaje C++ en el entorno Arduino IDE.

3.4 Software

3.5 Interfaz de usuario

La interfaz de usuario se programó con PyQt6, mediante un prompt de la inteligencia artificial (IA) Claude, modelo Sonnet 5 Esfuerzo Medio. Se alimentó del software original, que tenía la lógica de medición funcionando, pero que corría secuencialmente y se comunicaba con el usuario a través de la consola.

El prompt inicial fue: "Quiero hacer una interfaz de usuario sencilla que me permita comunicarme con este script. que se pueda elegir frecuencias y paso, visualizar modulo y fase de S11 y abaco de smith, que permita hacer calibración. Qué plataforma/lenguaje recomiendas, y cuanto tiempo tardaría en ponerla en funcionamiento?". La IA recomendó Python + PyQt6/PySide6, con matplotlib embebido para los gráficos. Dió las razones (reutilizar funciones, permite actualizar el plot en vivo, aprovecha librerías de Python para graficar) y explicó otras alternativas y por qué las descartó. Preguntó si hacía un esqueleto de la app con ventana principal, campos de frecuencia, threading de barrido y gráficos, y se le dió el permiso. Creó dos archivos, un "core" que contiene la misma lógica de cálculo que el código original pero reorganizada en funciones independientes (comunicación serial con Arduino, capura de audio, procesamiento de señal, detección del tono de batido y cálculo de S11 y álgebra de calibración) y un "GUI" (Graphical User Interface) que contiene la parte visual: ventana, botones, campos y gráficos. Este llama a las funciones implementadas en el core.

Se agregó la biblioteca PyQt6 para usar ventanas, botones, campos e hilos de ejecución. Se creó una clase (SweepWorker(QThread)) que se encarga de correr el barrido en segundo plano y avisar a la ventana qué se midió para que actualice los gráficos y la barra de progreso sin congelarse entre cada medición.

La IA generó además el diagrama de flujo que se observa en la figura 3.2. Los códigos se encuentran en el capítulo "Esquemáticos y código" 6.2.1.

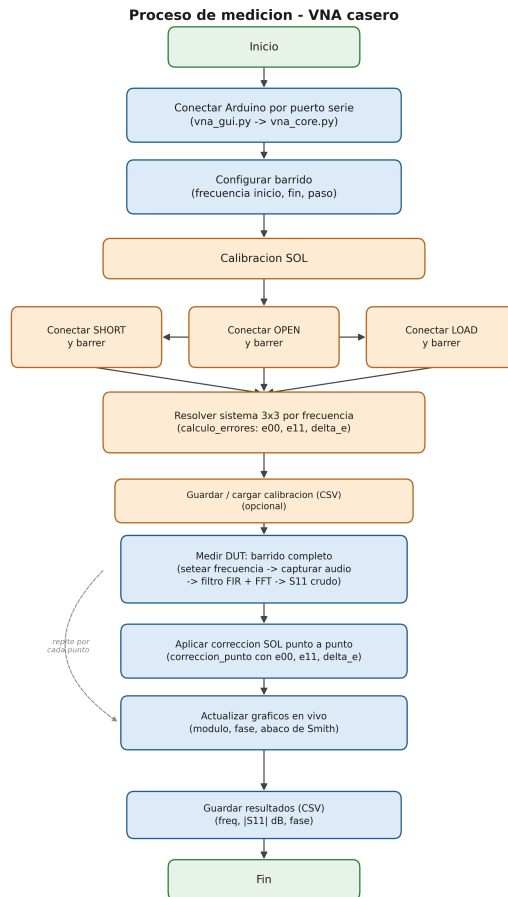


Figura 3.2: Diagrama de flujo de la interfaz de usuario

Capítulo 4

Resultados

Capítulo 5

Conclusiones y trabajos futuros

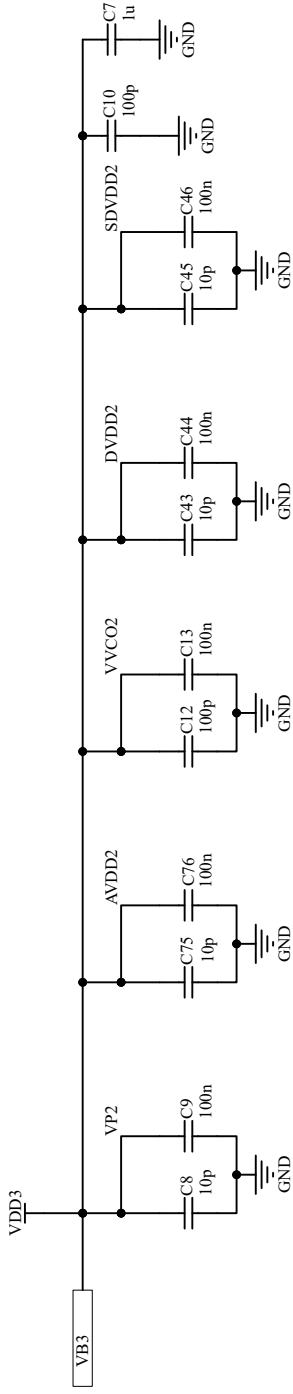
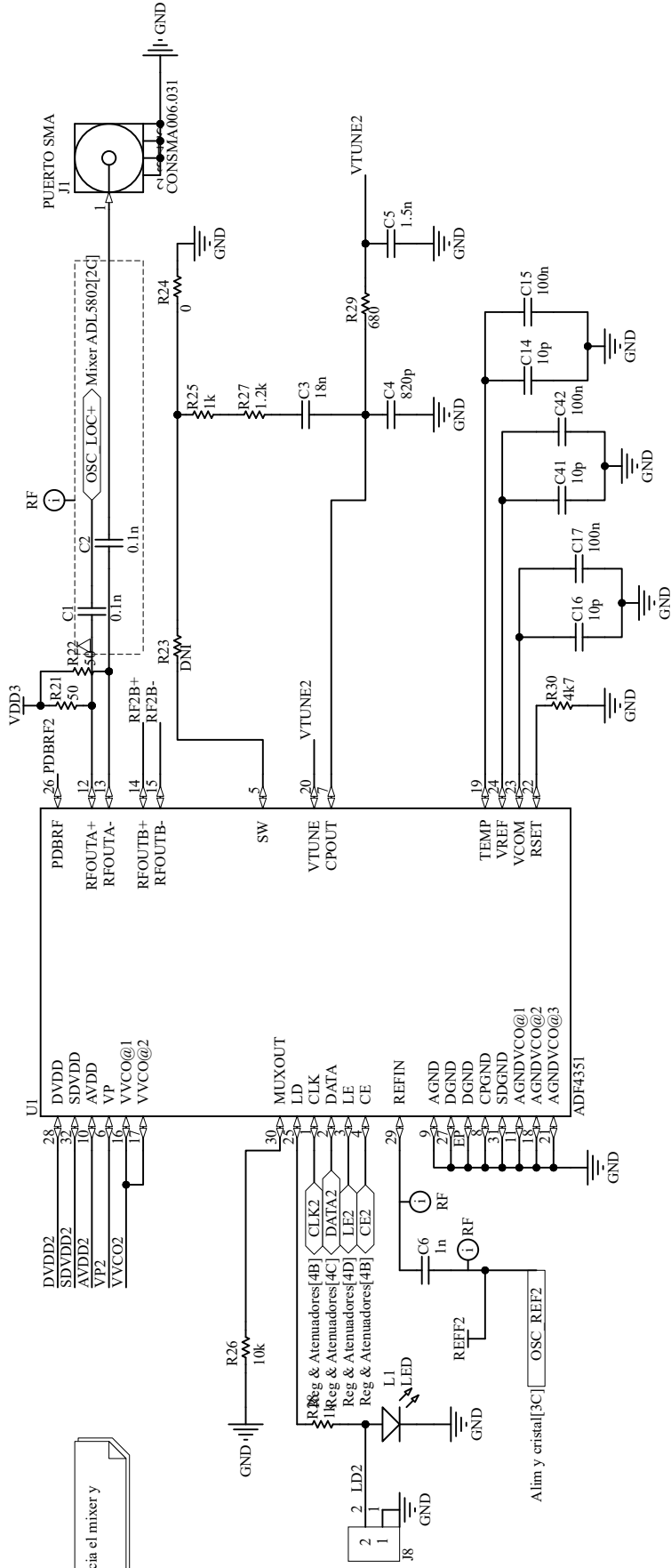
Capítulo 6

Esquemáticos y código

6.1 Esquemáticos Altium

A continuación se presentan los esquemáticos de los distintos bloques funcionales del sistema, exportados de Altium Designer.

Generrador desfasado FI, va hacia el mixer y actúa de oscilador local (OL)



poner capacitores de desacople cerca de los pines: 6 (Vp), 10 (AVDD), 16, 17 (VVCO), 19 (TEMP), 23 (VCOM), 24 (VREF), 28 DVDD, 32 SDVDD

Title ADF4351 (OL) derecho

Revision

Size A4

Date: 10/02/2026

File: C:\Users\...\ADF4351_der\SchDoc

Sheet of

Drawn By:

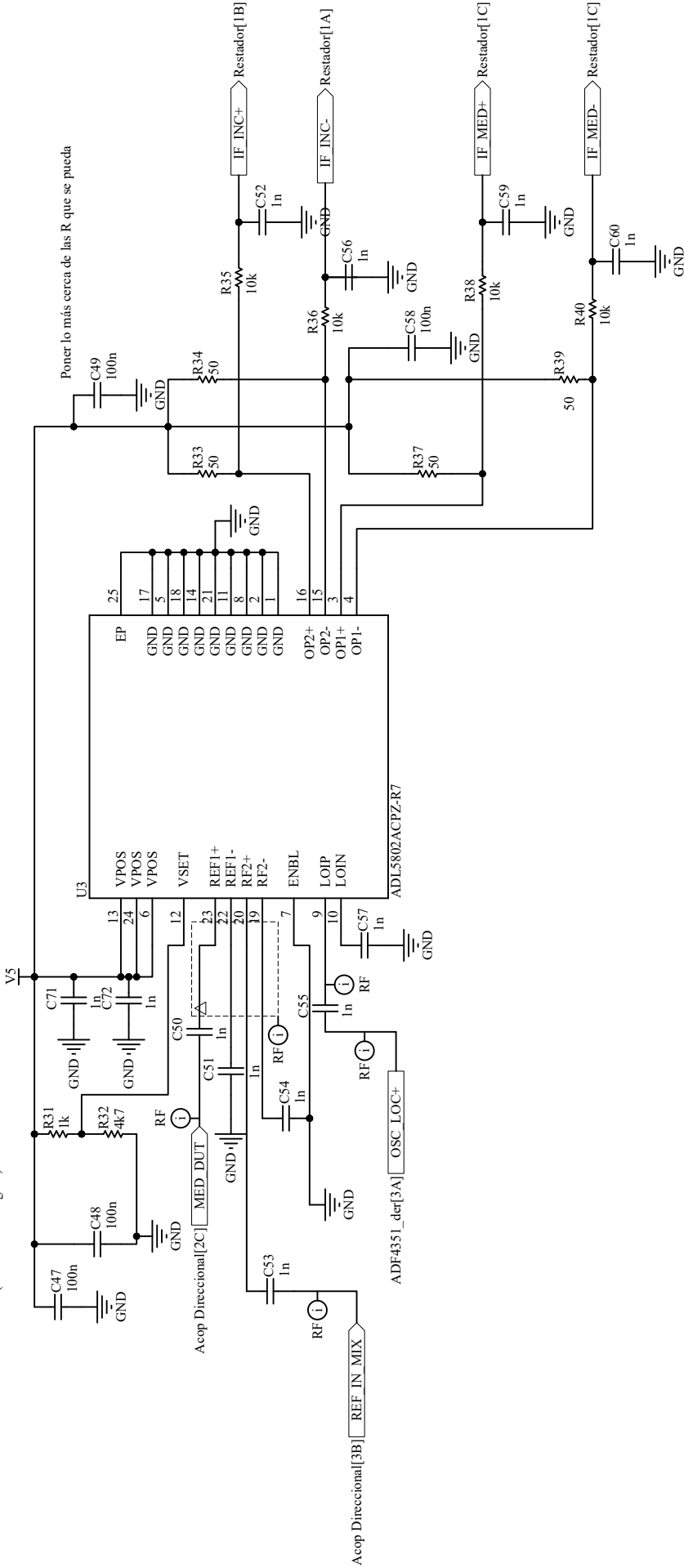
A

B

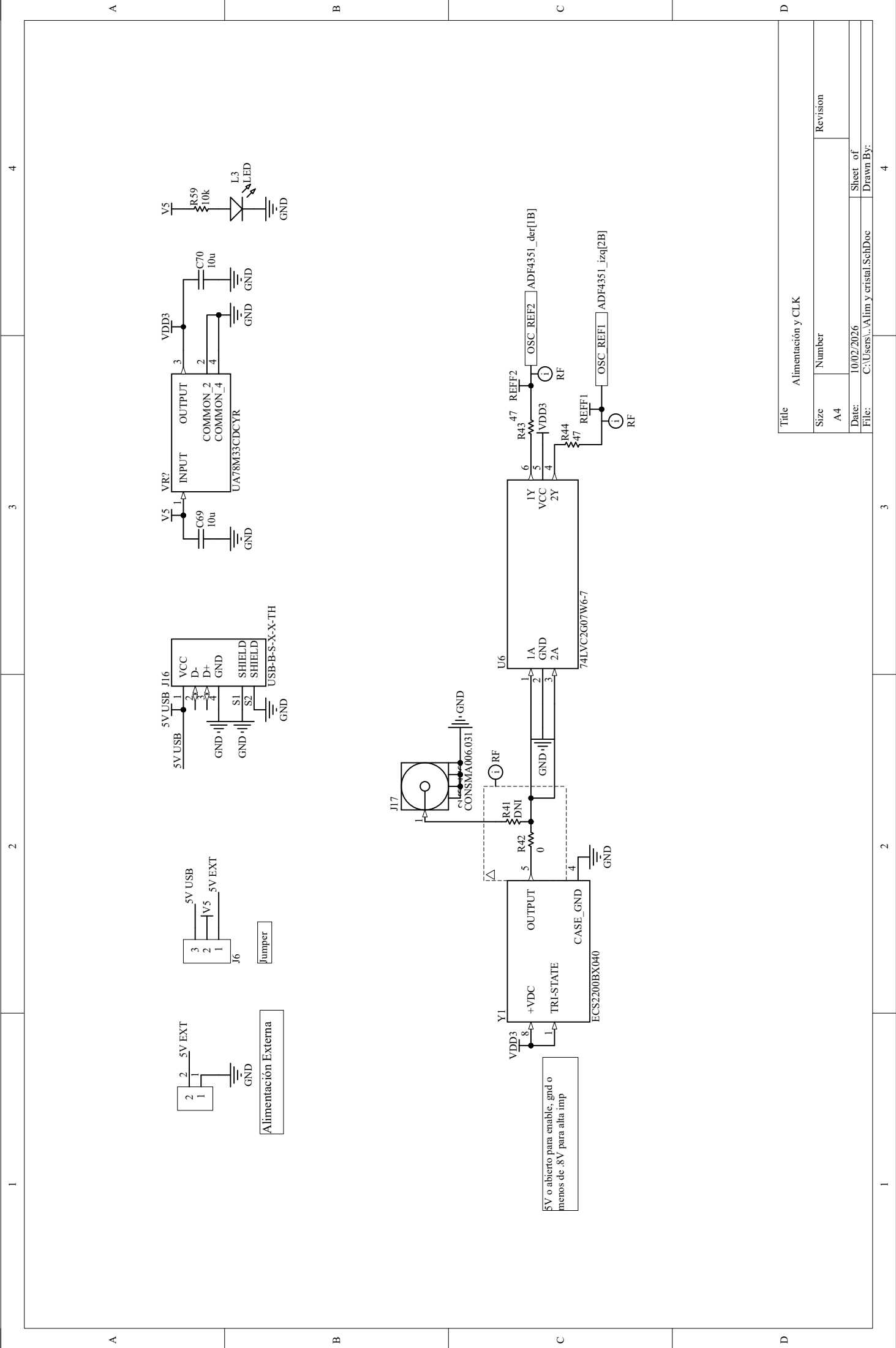
C

D

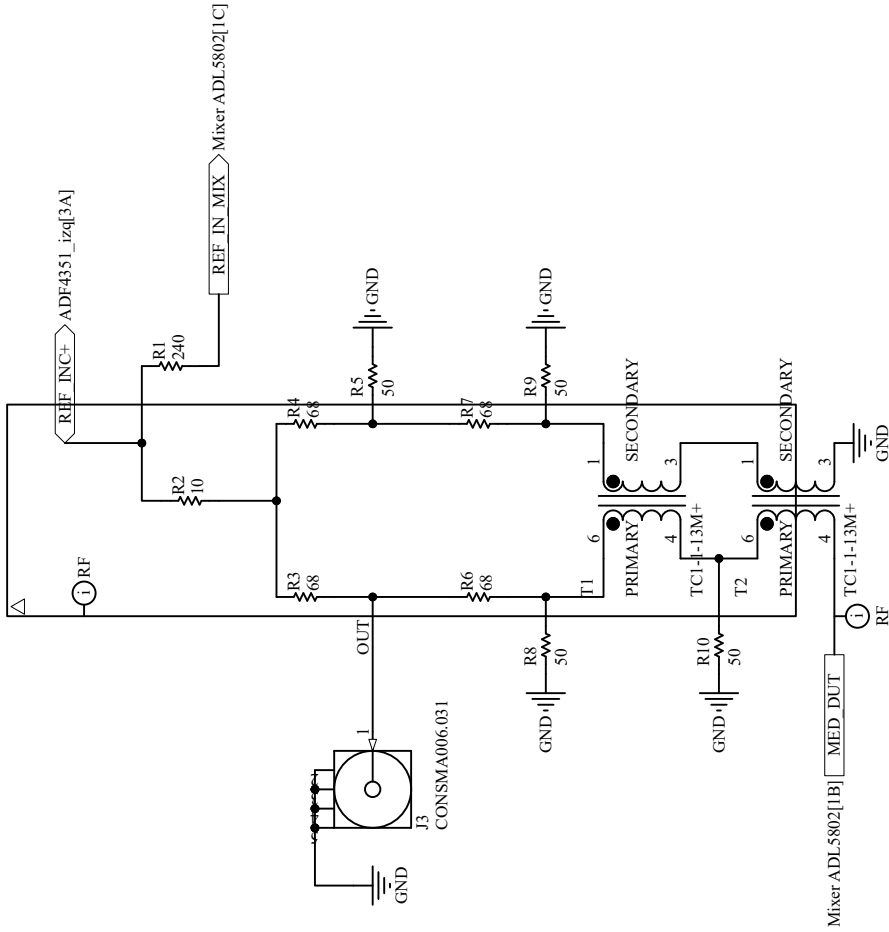
(ver IP3 noise figure) datasheet



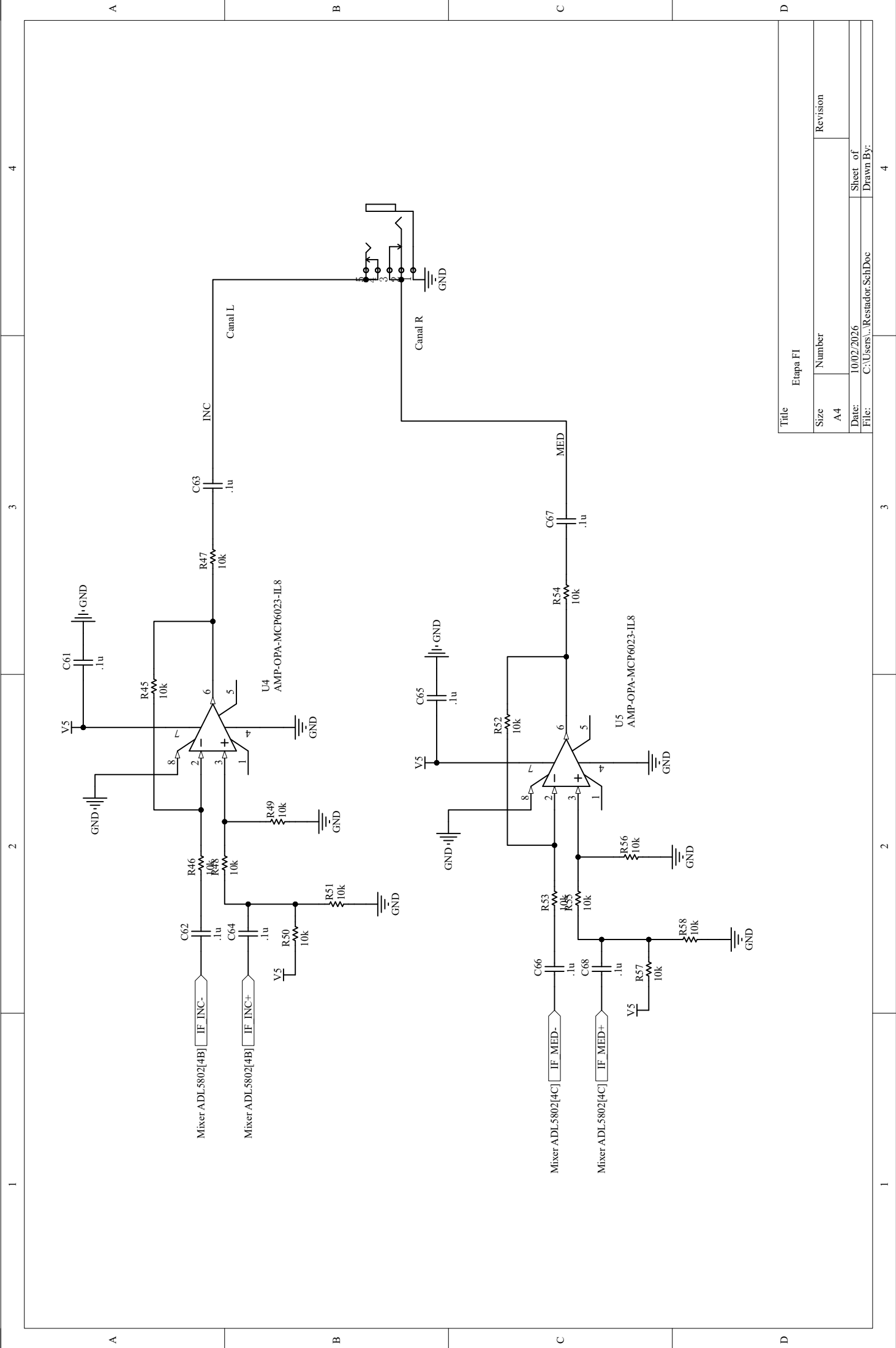
Title		Mezclador	
Size	Number	Revision	
A4			
Date:	10/02/2026	Sheet of	
File:	C:\Users\...\Mixer ADL5802.SchDoc	Drawn By:	



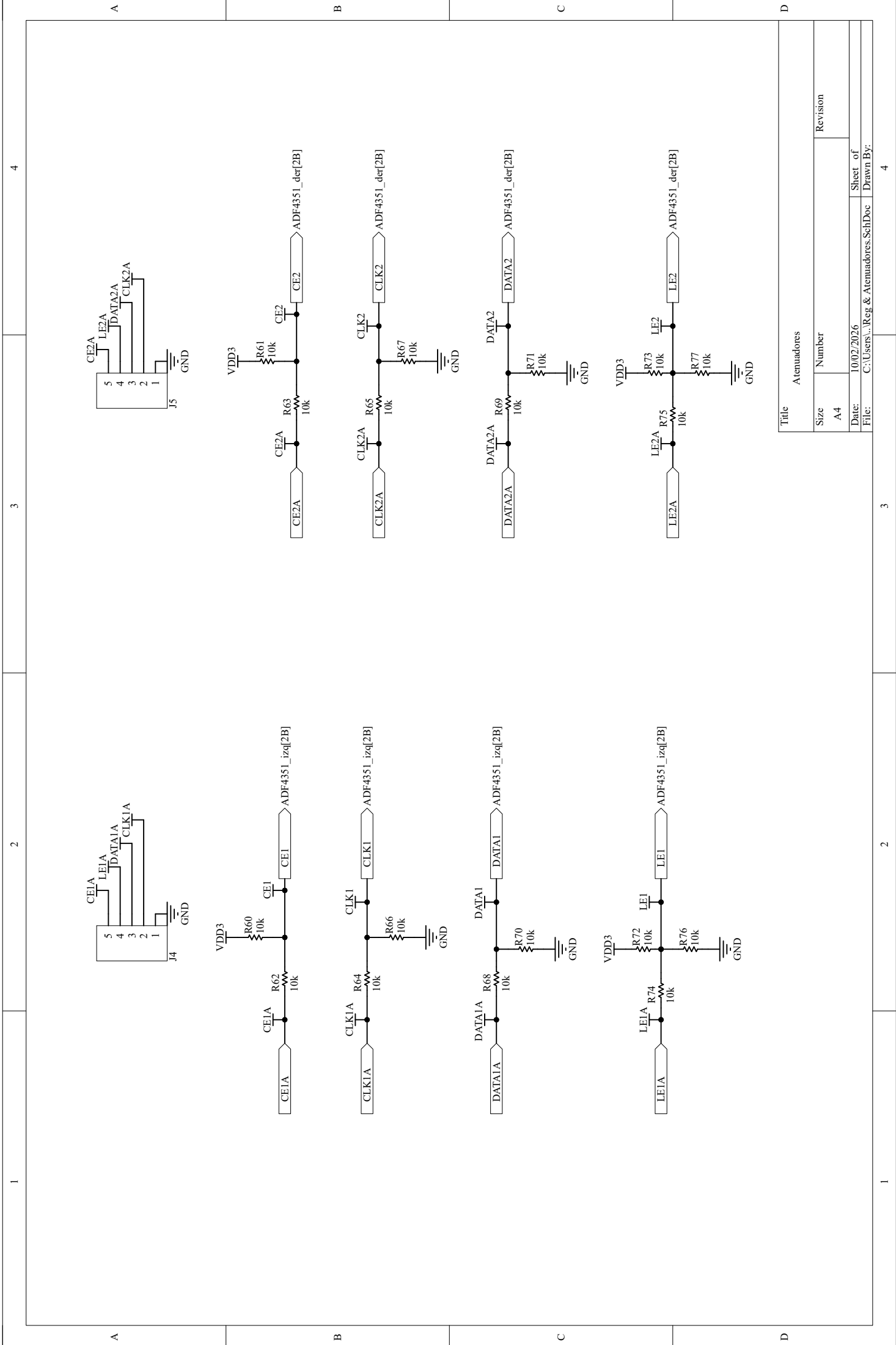
Title			
Alimentación y CLK			
Size	Number		Revision
A4			
Date:	10/02/2026		Sheet of
File:	C:\Users\...Alim y cristal.SchDoc		Drawn By:



Title Acoplador Direccional			
Size A4	Number		Revision
Date: 10/02/2026	Sheet of		Drawn By:
File: C:\Users\...\Acop Direccional.SchDoc	4		



Title		Etapa FI	
Size	Number	Revision	
A4			
Date:	10/02/2026	Sheet of	
File:	C:\Users\...\Restador.SchDoc	Drawn By:	



6.2 Interfaz de usuario

6.2.1 Core

```
1  """
2  una_core.py
3  =====
4  Nucleo de adquisicion y calculo para el VNA casero (2x ADF4351 + Arduino +
5  placa de sonido). Adaptado de VNA_calibrado.py para poder ser llamado desde
6  una interfaz grafica: sin input()/tkinter bloqueantes, con callback de
7  progreso por punto y posibilidad de abortar un barrido en curso.
8
9  La logica de senales (filtro FIR, FFT, deteccion del tono de batido,
10 algebra de calibracion SOL) es la misma que la del script original; lo
11 unico que cambia es como se invoca.
12
13 Requisitos:
14     pip install pyserial sounddevice numpy scipy
15 """
16
17 import csv
18 import time
19
20 import numpy as np
21 import serial
22 import sounddevice as sd
23 from scipy.signal import firwin, lfilter
24 from scipy.signal.windows import get_window
25
26
27 # ===== Lista por defecto de frecuencias excluidas
28 ↪ =====
29 # (copiada de VNA_calibrado.py; se puede editar/reemplazar desde la GUI)
30
31 FRECUENCIAS_EXCLUIDAS_MHZ_DEFAULT = [
32     200.00, 204.00, 208.00, 212.00, 216.00, 217.00, 221.00, 226.00, 256.00,
33     ↪ 260.00, 280.00,
34     285.00, 291.00, 320.00, 347.00, 353.00, 354.00, 359.00, 360.00, 365.00,
35     ↪ 366.00, 371.00,
36     372.00, 378.00, 379.00, 385.00, 386.00, 392.00, 393.00, 400.00, 401.00,
37     ↪ 408.00, 409.00,
38     416.00, 417.00, 424.00, 425.00, 433.00, 434.00, 443.00, 452.00, 512.00,
39     ↪ 559.00, 560.00,
40     570.00, 571.00, 581.00, 582.00, 617.00, 640.00, 678.00, 694.00, 705.00,
41     ↪ 706.00, 707.00,
42     708.00, 717.00, 718.00, 719.00, 720.00, 730.00, 731.00, 732.00, 742.00,
43     ↪ 743.00, 744.00,
44     745.00, 756.00, 757.00, 758.00, 759.00, 770.00, 771.00, 772.00, 773.00,
45     ↪ 784.00, 785.00,
46     786.00, 787.00, 799.00, 800.00, 801.00, 802.00, 815.00, 816.00, 817.00,
47     ↪ 818.00, 831.00,
48     832.00, 833.00, 834.00, 848.00, 849.00, 850.00, 851.00, 865.00, 866.00,
49     ↪ 867.00, 868.00,
```

```

40     883.00, 886.00, 904.00, 927.00, 929.00, 955.00, 1009.00, 1024.00, 1075.00,
    ↪ 1118.00, 1119.00,
41     1120.00, 1140.00, 1142.00, 1162.00, 1164.00, 1185.00, 1187.00, 1234.00,
    ↪ 1280.00, 1283.00,
42     1304.00, 1333.00, 1356.00, 1383.00, 1388.00, 1410.00, 1411.00, 1412.00,
    ↪ 1413.00, 1414.00,
43     1415.00, 1416.00, 1434.00, 1435.00, 1436.00, 1437.00, 1438.00, 1439.00,
    ↪ 1440.00, 1459.00,
44     1460.00, 1461.00, 1462.00, 1463.00, 1464.00, 1465.00, 1484.00, 1485.00,
    ↪ 1486.00, 1487.00,
45     1488.00, 1489.00, 1490.00, 1491.00, 1511.00, 1512.00, 1513.00, 1514.00,
    ↪ 1515.00, 1516.00,
46     1517.00, 1518.00, 1539.00, 1540.00, 1541.00, 1542.00, 1543.00, 1544.00,
    ↪ 1545.00, 1546.00,
47     1568.00, 1569.00, 1570.00, 1571.00, 1572.00, 1573.00, 1574.00, 1575.00,
    ↪ 1598.00, 1599.00,
48     1600.00, 1601.00, 1602.00, 1603.00, 1604.00, 1605.00, 1629.00, 1630.00,
    ↪ 1631.00, 1632.00,
49     1633.00, 1634.00, 1635.00, 1636.00, 1662.00, 1663.00, 1664.00, 1665.00,
    ↪ 1666.00, 1667.00,
50     1668.00, 1669.00, 1696.00, 1697.00, 1698.00, 1700.00, 1701.00, 1702.00,
    ↪ 1730.00, 1731.00,
51     1732.00, 1734.00, 1735.00, 1736.00, 1766.00, 1767.00, 1771.00, 1772.00,
    ↪ 1803.00, 1829.00,
52     1833.00, 1854.00, 1858.00, 1910.00, 1937.00, 2018.00, 2150.00, 2235.00,
    ↪ 2236.00,
53     2237.00, 2238.00, 2239.00, 2240.00, 2241.00, 2279.00, 2280.00, 2281.00,
    ↪ 2283.00, 2284.00,
54     2324.00, 2325.00, 2327.00, 2328.00, 2329.00, 2370.00, 2374.00, 2421.00,
    ↪ 2463.00,
55 ]
56
57
58 # ===== Configuración por defecto =====
59
60 def config_por_defecto():
61     """Devuelve un dict de configuración editable desde la GUI."""
62     return {
63         "fs_audio": 44100,
64         "duracion_captura_s": 0.6,
65         "recorte_inicial_s": 0.150,
66         "dispositivo_audio": None,
67         "f_centro_filtro": 10000,      # Hz, tono de batido esperado
68         "ancho_banda_filtro": 200,    # Hz
69         "numtaps_filtro": 1001,
70         "ventana_fft": "flattop",
71         "frecuencias_excluidas": list(FRECUENCIAS_EXCLUIDAS_MHZ_DEFAULT),
72     }
73
74
75 def banda_busqueda_pico(config):
76     fc = config["f_centro_filtro"]

```

```

77     bw = config["ancho_banda_filtro"]
78     return (fc - bw / 2, fc + bw / 2)
79
80
81 # ===== Arduino / serial =====
82
83 class ErrorArduino(Exception):
84     pass
85
86
87 def conectar_arduino(puerto, baudrate=115200, timeout=5):
88     """Abre el puerto serie y devuelve (objeto_serial,
89     ↪ primera_linea_recibida)."""
90     ser = serial.Serial(puerto, baudrate, timeout=timeout)
91     time.sleep(2) # el Arduino se resetea al abrir el puerto; esperamos a que
92     ↪ arranque
93     ser.reset_input_buffer()
94     linea = ser.readline().decode(errors="ignore").strip()
95     return ser, linea
96
97 def setear_frecuencia(ser, f_mhz):
98     """Manda la frecuencia al Arduino y espera la confirmacion (OK/ERROR)."""
99     comando = f"{f_mhz:.6f}\n"
100     ser.write(comando.encode())
101     respuesta = ser.readline().decode(errors="ignore").strip()
102     intentos_vacios = 0
103     while respuesta and not respuesta.startswith("OK") and not
104     ↪ respuesta.startswith("ERROR"):
105         respuesta = ser.readline().decode(errors="ignore").strip()
106         intentos_vacios += 1
107         if intentos_vacios > 50:
108             break
109     return respuesta
110
111 def generar_frecuencias(inicio, fin, paso):
112     """Genera la lista de frecuencias evitando arrastre de error de punto
113     ↪ flotante."""
114     if paso <= 0:
115         raise ValueError("El paso de frecuencia debe ser mayor que 0.")
116     n_pasos = round((fin - inicio) / paso)
117     return [inicio + i * paso for i in range(n_pasos + 1)]
118
119 # ===== Adquisicion de audio =====
120
121 def capturar_audio(duracion_s, fs, dispositivo=None, recorte_inicial_s=0.150):
122     """
123     Graba desde la entrada configurada. Devuelve (canal_ref, canal_med) como
124     arrays float64, con canal_ref = izquierdo, canal_med = derecho.

```

```

125     n_muestras = int(duracion_s * fs)
126     grabacion_con_ruido = sd.rec(
127         n_muestras, samplerate=fs, channels=2, dtype="float64",
128         ↪ device=dispositivo
129     )
130     sd.wait()
131     grabacion = grabacion_con_ruido[int(fs * recorte_inicial_s):]
132     canal_ref = grabacion[:, 0]
133     canal_med = grabacion[:, 1]
134     return canal_ref, canal_med
135
136     # ===== Filtro FIR + FFT + S11 =====
137
138     def diseñar_filtro_pasabanda(fs, f_centro, ancho_banda, numtaps,
139     ↪ duracion_captura_s):
140         n_muestras_captura = int(duracion_captura_s * fs)
141         if numtaps >= n_muestras_captura:
142             raise ValueError(
143                 f"NUMTAPS_FILTRO ({numtaps}) debe ser menor que la cantidad de "
144                 f"muestras por captura ({n_muestras_captura}, con
145                 ↪ duracion_captura_s="
146                 f"{duracion_captura_s}s y fs={fs}Hz). Bajar numtaps o subir la
147                 ↪ duracion."
148             )
149         f_baja = f_centro - ancho_banda / 2
150         f_alta = f_centro + ancho_banda / 2
151         return firwin(numtaps, [f_baja, f_alta], pass_zero=False, fs=fs,
152             ↪ window="hamming")
153
154     def aplicar_filtro(señal, coeficientes):
155         return lfilter(coeficientes, 1.0, señal)
156
157     def calcular_espectro_complejo(señal, fs, ventana):
158         n = len(señal)
159         w = get_window(ventana, n)
160         fft_vals = np.fft.fft(señal * w)
161         fft_freqs = np.fft.fftfreq(n, d=1 / fs)
162         mitad = n // 2
163         return fft_freqs[:mitad], fft_vals[:mitad]
164
165     def buscar_indice_pico(freqs, espectro, banda):
166         """Devuelve el índice del bin de mayor magnitud dentro de la banda dada."""
167         f_baja, f_alta = banda
168         i0 = np.searchsorted(freqs, f_baja)
169         i1 = np.searchsorted(freqs, f_alta)
170         if i1 <= i0:
171             raise ValueError("La banda de búsqueda no contiene ningún bin de
172             ↪ frecuencia.")

```

```

171     magnitud = np.abs(espectro[i0:i1])
172     return i0 + np.argmax(magnitud)
173
174
175 def medir_s11_punto(canal_ref, canal_med, fs, coef_filtro, ventana_fft, banda):
176     """
177     A partir de una captura de referencia y medicion, devuelve un unico
178     valor complejo de S11 (sin corregir) correspondiente al tono de batido,
179     y la frecuencia de batido detectada (Hz).
180     """
181     ref_filtrada = aplicar_filtro(canal_ref, coef_filtro)
182     med_filtrada = aplicar_filtro(canal_med, coef_filtro)
183
184     freqs_ref, espectro_ref = calcular_espectro_complejo(ref_filtrada, fs,
185     ↪     ventana_fft)
186     _, espectro_med = calcular_espectro_complejo(med_filtrada, fs, ventana_fft)
187
188     idx_pico = buscar_indice_pico(freqs_ref, espectro_ref, banda)
189
190     epsilon = np.max(np.abs(espectro_ref)) * 1e-9
191     s11 = np.conj(espectro_med[idx_pico] / (espectro_ref[idx_pico] + epsilon))
192
193     return s11, freqs_ref[idx_pico]
194
195 # ===== Barrido (con callback de progreso + aborto)
196 ↪ =====
197
198 class BarridoAbortado(Exception):
199     """Se lanza cuando should_abort() devuelve True durante un barrido."""
200     pass
201
202 def barrido(ser, frecuencias_rf, coef_filtro, config, on_point=None,
203     ↪     should_abort=None):
204     """
205     Ejecuta un barrido completo de frecuencias, midiendo un punto de S11
206     (sin corregir) por cada frecuencia.
207
208     on_point(i, n, f_rf, s11, f_batido_hz): callback opcional, llamado
209     ↪     despues de medir cada punto (i es 1-based).
210     should_abort(): callable opcional; si devuelve True se aborta el
211     ↪     barrido lanzando BarridoAbortado.
212
213     Devuelve (freqs_medidas, s11_medidos) como arrays de numpy, salteando
214     ↪     las frecuencias que esten en config["frecuencias_excluidas"].
215     """
216     resultados_freq_rf = []
217     resultados_s11 = []
218     excluidas = config.get("frecuencias_excluidas", [])
219     banda = banda_búsqueda_pico(config)

```

```

220     for i, f_rf in enumerate(frecuencias_rf):
221         if should_abort is not None and should_abort():
222             raise BarridoAbortado()
223
224         if any(abs(f_rf - fx) < 1e-6 for fx in excluidas):
225             continue
226
227         respuesta = setear_frecuencia(ser, f_rf)
228         if respuesta.startswith("ERROR"):
229             raise ErrorArduino(f"Arduino devolvio error en {f_rf:.3f} MHz:
230                 ↳ {respuesta}")
231
232         canal_ref, canal_med = capturar_audio(
233             config["duracion_captura_s"], config["fs_audio"],
234             ↳ config.get("dispositivo_audio"),
235             config.get("recorte_inicial_s", 0.150),
236         )
237         s11, f_batido = medir_s11_punto(
238             canal_ref, canal_med, config["fs_audio"], coef_filtro,
239             config["ventana_fft"], banda,
240         )
241
242         resultados_freq_rf.append(f_rf)
243         resultados_s11.append(s11)
244
245         if on_point is not None:
246             on_point(i + 1, len(frecuencias_rf), f_rf, s11, f_batido)
247
248     return np.array(resultados_freq_rf), np.array(resultados_s11)
249
250 # ===== Calibracion SOL =====
251
252 def calculo_errores(GM_CC, GM_CA, GM_50, n):
253     """Resuelve, punto a punto, el sistema 3x3 de la calibracion SOL."""
254     e_00 = np.empty(n, dtype=complex)
255     e_11 = np.empty(n, dtype=complex)
256     delta_e = np.empty(n, dtype=complex)
257
258     for i in range(n):
259         A = np.array([
260             [1, -GM_CC[i], 1], # CC (short)
261             [1, 0, 0], # 50 (load)
262             [1, GM_CA[i], -1], # CA (open)
263         ], dtype=complex)
264         B = np.array([GM_CC[i], GM_50[i], GM_CA[i]])
265         sol = np.linalg.solve(A, B)
266         e_00[i] = sol[0]
267         e_11[i] = sol[1]
268         delta_e[i] = sol[2]
269
270     return e_00, e_11, delta_e

```

```

270
271
272 def correccion(GM, e_00, e_11, delta_e, n):
273     """Aplica la correccion de error de la calibracion SOL a un array de GM
    ↪ medidos."""
274     Gamma = np.empty(n, dtype=complex)
275     for i in range(n):
276         denominador = (GM[i] * e_11[i] - delta_e[i])
277         if abs(denominador) < 1e-15:
278             denominador = 1e-15 + 1e-15j
279         Gamma[i] = (GM[i] - e_00[i]) / denominador
280     return Gamma
281
282
283 def correccion_punto(gm, e_00_i, e_11_i, delta_e_i):
284     """Version de correccion() para un unico punto (usada para graficar en
    ↪ vivo)."""
285     denominador = gm * e_11_i - delta_e_i
286     if abs(denominador) < 1e-15:
287         denominador = 1e-15 + 1e-15j
288     return (gm - e_00_i) / denominador
289
290
291 def guardar_calibracion_csv(path, freqs, e_00, e_11, delta_e):
292     with open(path, "w", newline="", encoding="utf-8") as f:
293         writer = csv.writer(f)
294         writer.writerow(["freq_MHz", "e_00_real", "e_00_imag", "e_11_real",
    ↪ "e_11_imag",
295                             "delta_e_real", "delta_e_imag"])
296         for i in range(len(freqs)):
297             writer.writerow([
298                 f"{freqs[i]:.6f}",
299                 f"{e_00[i].real:.10e}", f"{e_00[i].imag:.10e}",
300                 f"{e_11[i].real:.10e}", f"{e_11[i].imag:.10e}",
301                 f"{delta_e[i].real:.10e}", f"{delta_e[i].imag:.10e}",
302             ])
303
304
305 def cargar_calibracion_csv(path):
306     """Rearma los valores complejos guardados por guardar_calibracion_csv()."""
307     freqs, e_00, e_11, delta_e = [], [], [], []
308     with open(path, "r", encoding="utf-8") as f:
309         reader = csv.DictReader(f)
310         for row in reader:
311             freqs.append(float(row["freq_MHz"]))
312             e_00.append(complex(float(row["e_00_real"]),
    ↪ float(row["e_00_imag"])))
313             e_11.append(complex(float(row["e_11_real"]),
    ↪ float(row["e_11_imag"])))
314             delta_e.append(complex(float(row["delta_e_real"]),
    ↪ float(row["delta_e_imag"])))
315     return np.array(freqs), np.array(e_00), np.array(e_11), np.array(delta_e)

```

```

316
317
318 def guardar_resultados_csv(path, freq_rf, modulo_db, fase_deg):
319     with open(path, "w", newline="", encoding="utf-8") as f:
320         writer = csv.writer(f)
321         writer.writerow(["freq_rf_MHz", "modulo_S11_dB", "fase_S11_deg"])
322         for f_rf, m, fa in zip(freq_rf, modulo_db, fase_deg):
323             writer.writerow([f"{f_rf:.6f}", f"{m:.4f}", f"{fa:.4f}"])
324
325
326 def modulo_fase(s11_array):
327     """Convierte un array de S11 complejos a (modulo_dB, fase_grados)."""
328     modulo_db = 20 * np.log10(np.abs(s11_array) + 1e-12)
329     fase_deg = np.degrees(np.angle(s11_array))
330     return modulo_db, fase_deg

```

6.2.2 Graphical User Interface

```

1  """
2  vna_gui.py
3  =====
4  Interfaz grafica (PyQt6) para el VNA casero: permite elegir frecuencia de
5  inicio/fin/paso, conectar el Arduino, correr la calibracion SOL (short /
6  open / load) paso a paso, medir un DUT, y visualizar en vivo el modulo y
7  la fase de S11 junto con el abaco de Smith.
8
9  Requisitos:
10     pip install PyQt6 pyserial sounddevice numpy scipy matplotlib
11
12  Ejecutar con:
13     python vna_gui.py
14  """
15
16 import os
17 import sys
18
19 import numpy as np
20
21 from PyQt6.QtCore import Qt, QThread, pyqtSignal
22 from PyQt6.QtWidgets import (
23     QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
24     QGroupBox, QLabel, QLineEdit, QPushButton, QDoubleSpinBox,
25     QProgressBar, QFileDialog, QMessageBox, QPlainTextEdit, QSplitter,
26 )
27
28 from matplotlib.backends.backend_qtagg import FigureCanvasQTagg as FigureCanvas
29 from matplotlib.figure import Figure
30
31 import vna_core as core
32
33
34 # ===== Hilo de trabajo: corre un barrido sin trabar la GUI
↪  =====

```

```

35
36 class SweepWorker(QThread):
37     """Ejecuta core.barrido() en un hilo aparte y reenvia sus resultados como
    ↪ señales Qt."""
38
39     punto_medido = pyqtSignal(int, int, float, complex, float) # i, n, f_rf,
    ↪ s11, f_batido
40     terminado = pyqtSignal(object, object) # freqs
    ↪ (ndarray), s11 (ndarray)
41     abortado = pyqtSignal()
42     error = pyqtSignal(str)
43
44     def __init__(self, ser, frecuencias_rf, coef_filtro, config, parent=None):
45         super().__init__(parent)
46         self.ser = ser
47         self.frecuencias_rf = frecuencias_rf
48         self.coef_filtro = coef_filtro
49         self.config = config
50         self._abortar = False
51
52     def abortar(self):
53         self._abortar = True
54
55     def run(self):
56         try:
57             freqs, s11 = core.barrido(
58                 self.ser, self.frecuencias_rf, self.coef_filtro, self.config,
59                 on_point=lambda i, n, f, s, fb: self.punto_medido.emit(i, n, f,
    ↪ s, fb),
60                 should_abort=lambda: self._abortar,
61             )
62             self.terminado.emit(freqs, s11)
63         except core.BarridoAbortado:
64             self.abortado.emit()
65         except Exception as e: # noqa: BLE001 - se muestra en la GUI
66             self.error.emit(str(e))
67
68
69     # ===== Dibujo del abaco de Smith (fondo estatico)
    ↪ =====
70
71     def dibujar_fondo_smith(ax):
72         """Dibuja el grillado clasico de un abaco de Smith (r y x constantes)."""
73         ax.clear()
74         ax.set_aspect("equal")
75         ax.axis("off")
76         ax.set_xlim(-1.15, 1.15)
77         ax.set_ylim(-1.15, 1.15)
78         ax.set_title("Abaco de Smith")
79
80         theta = np.linspace(0, 2 * np.pi, 400)
81

```

```

82     # Circulo unitario ( $|\Gamma| = 1$ )
83     ax.plot(np.cos(theta), np.sin(theta), color="black", linewidth=1.0)
84     ax.axhline(0, color="gray", linewidth=0.5)
85
86     # Circulos de resistencia constante ( $r = 0, 0.2, 0.5, 1, 2, 5$ )
87     for r in (0.0, 0.2, 0.5, 1.0, 2.0, 5.0):
88         cx = r / (r + 1)
89         rad = 1 / (r + 1)
90         x = cx + rad * np.cos(theta)
91         y = rad * np.sin(theta)
92         ax.plot(x, y, color="gray", linewidth=0.5)
93
94     # Arcos de reactancia constante ( $x = +/- 0.2, 0.5, 1, 2, 5$ )
95     t = np.linspace(0, 2 * np.pi, 800)
96     for x_val in (0.2, 0.5, 1.0, 2.0, 5.0):
97         for signo in (1, -1):
98             xv = signo * x_val
99             cx, cy = 1.0, 1.0 / xv
100             rad = abs(1.0 / xv)
101             gx = cx + rad * np.cos(t)
102             gy = cy + rad * np.sin(t)
103             mask = gx ** 2 + gy ** 2 <= 1.0 + 1e-6
104             ax.plot(gx[mask], gy[mask], color="gray", linewidth=0.5)
105
106
107     # ===== Ventana principal =====
108
109     class MainWindow(QMainWindow):
110         def __init__(self):
111             super().__init__()
112             self.setWindowTitle("VNA - Control y medición de S11")
113             self.resize(1200, 750)
114
115             self.config = core.config_por_defecto()
116             self.ser = None
117             self.hilo = None
118
119             # Estado de calibracion
120             self.cal_paso = 0          # 0 = sin empezar, 1=SHORT, 2=OPEN, 3=LOAD,
121             ↪ 4=lista
122             self.cal_gm_cc = None
123             self.cal_gm_ca = None
124             self.cal_gm_50 = None
125             self.cal_freqs = None
126             self.cal_e00 = None
127             self.cal_e11 = None
128             self.cal_delta_e = None
129             self.cal_disponible = False
130
131             # Buffers de la medicion en curso (para graficar en vivo)
132             self.buf_freq = []
133             self.buf_mod_db = []

```

```

133     self.buf_fase_deg = []
134     self.buf_gamma = []
135     self.ultima_medicion_freqs = None
136     self.ultima_medicion_gamma = None
137     self.ultima_medicion_mod_db = None
138     self.ultima_medicion_fase_deg = None
139
140     self._armar_ui()
141
142     # ----- UI -----
143
144     def _armar_ui(self):
145         central = QWidget()
146         self.setCentralWidget(central)
147         layout_principal = QHBoxLayout(central)
148
149         splitter = QSplitter(Qt.Orientation.Horizontal)
150         layout_principal.addWidget(splitter)
151
152         panel_izq = self._armar_panel_controles()
153         panel_der = self._armar_panel_graficos()
154
155         splitter.addWidget(panel_izq)
156         splitter.addWidget(panel_der)
157         splitter.setStretchFactor(0, 0)
158         splitter.setStretchFactor(1, 1)
159
160     def _armar_panel_controles(self):
161         panel = QWidget()
162         panel.setMinimumWidth(340)
163         panel.setMaximumWidth(420)
164         v = QVBoxLayout(panel)
165
166         # --- Conexion Arduino ---
167         gb_conexion = QGroupBox("Conexion Arduino")
168         f = QVBoxLayout(gb_conexion)
169         fila_puerto = QHBoxLayout()
170         fila_puerto.addWidget(QLabel("Puerto:"))
171         self.ed_puerto = QLineEdit("COM3")
172         fila_puerto.addWidget(self.ed_puerto)
173         f.addLayout(fila_puerto)
174
175         fila_baud = QHBoxLayout()
176         fila_baud.addWidget(QLabel("Baudrate:"))
177         self.ed_baudrate = QLineEdit("115200")
178         fila_baud.addWidget(self.ed_baudrate)
179         f.addLayout(fila_baud)
180
181         self.btn_conectar = QPushButton("Conectar")
182         self.btn_conectar.clicked.connect(self._conectar_arduino)
183         f.addWidget(self.btn_conectar)
184

```

```

185     self.lbl_estado_conexion = QLabel("Desconectado")
186     f.addWidget(self.lbl_estado_conexion)
187     v.addWidget.gb_conexion)
188
189     # --- Barrido ---
190     gb_barrido = QGroupBox("Barrido de frecuencia")
191     f2 = QVBoxLayout.gb_barrido)
192
193     self.sp_f_inicio = self._nuevo_spin(1800.0, 0, 6000, " MHz")
194     self.sp_f_fin = self._nuevo_spin(2200.0, 0, 6000, " MHz")
195     self.sp_f_paso = self._nuevo_spin(1.0, 0.001, 1000, " MHz")
196
197     f2.addWidget(self._fila("F inicio:", self.sp_f_inicio))
198     f2.addWidget(self._fila("F fin:", self.sp_f_fin))
199     f2.addWidget(self._fila("F paso:", self.sp_f_paso))
200     v.addWidget.gb_barrido)
201
202     # --- Calibracion SOL ---
203     gb_cal = QGroupBox("Calibracion SOL")
204     f3 = QVBoxLayout.gb_cal)
205
206     self.lbl_estado_cal = QLabel("Sin calibrar")
207     f3.addWidget(self.lbl_estado_cal)
208
209     self.btn_cal_iniciar = QPushButton("Iniciar calibracion nueva")
210     self.btn_cal_iniciar.clicked.connect(self._iniciar_calibracion)
211     f3.addWidget(self.btn_cal_iniciar)
212
213     self.btn_cal_paso = QPushButton("Barrer paso actual")
214     self.btn_cal_paso.setEnabled(False)
215     self.btn_cal_paso.clicked.connect(self._barrer_paso_calibracion)
216     f3.addWidget(self.btn_cal_paso)
217
218     self.btn_cal_cargar = QPushButton("Cargar calibracion desde archivo...")
219     self.btn_cal_cargar.clicked.connect(self._cargar_calibracion)
220     f3.addWidget(self.btn_cal_cargar)
221
222     self.btn_cal_guardar = QPushButton("Guardar calibracion actual...")
223     self.btn_cal_guardar.setEnabled(False)
224     self.btn_cal_guardar.clicked.connect(self._guardar_calibracion)
225     f3.addWidget(self.btn_cal_guardar)
226
227     v.addWidget.gb_cal)
228
229     # --- Medicion DUT ---
230     gb_dut = QGroupBox("Medición del DUT")
231     f4 = QVBoxLayout.gb_dut)
232
233     self.btn_medir = QPushButton("Medir DUT")
234     self.btn_medir.setEnabled(False)
235     self.btn_medir.clicked.connect(self._iniciar_medicion_dut)
236     f4.addWidget(self.btn_medir)

```

```

237
238     self.btn_abortar = QPushButton("Abortar barrido")
239     self.btn_abortar.setEnabled(False)
240     self.btn_abortar.clicked.connect(self._abortar_barrido)
241     f4.addWidget(self.btn_abortar)
242
243     self.progreso = QProgressBar()
244     f4.addWidget(self.progreso)
245
246     self.btn_guardar_resultados = QPushButton("Guardar resultados CSV...")
247     self.btn_guardar_resultados.setEnabled(False)
248     self.btn_guardar_resultados.clicked.connect(self._guardar_resultados)
249     f4.addWidget(self.btn_guardar_resultados)
250
251     v.addWidget(gb_dut)
252
253     # --- Log ---
254     gb_log = QGroupBox("Log")
255     f5 = QVBoxLayout(gb_log)
256     self.txt_log = QPlainTextEdit()
257     self.txt_log.setReadOnly(True)
258     f5.addWidget(self.txt_log)
259     v.addWidget(gb_log, stretch=1)
260
261     return panel
262
263 def _armar_panel_graficos(self):
264     panel = QWidget()
265     v = QVBoxLayout(panel)
266
267     # Figura módulo + fase (dos subplots apilados)
268     self.fig_mf = Figure(figsize=(6, 5), facecolor="#fad7ea")
269     self.ax_mod = self.fig_mf.add_subplot(211)
270     self.ax_fase = self.fig_mf.add_subplot(212, sharex=self.ax_mod)
271     self._preparar_ejes_mod_fase()
272     self.canvas_mf = FigureCanvas(self.fig_mf)
273
274     # Figura abaco de Smith
275     self.fig_smith = Figure(figsize=(5, 5), facecolor="#fad7ea")
276     self.ax_smith = self.fig_smith.add_subplot(111)
277     dibujar_fondo_smith(self.ax_smith)
278     (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
279                                             marker=".", linestyle="None")
280     self.canvas_smith = FigureCanvas(self.fig_smith)
281
282     fila_graficos = QHBoxLayout()
283     fila_graficos.addWidget(self.canvas_mf, stretch=1)
284     fila_graficos.addWidget(self.canvas_smith, stretch=1)
285     v.addLayout(fila_graficos)
286
287     return panel
288

```

```

289 def _preparar_ejes_mod_fase(self):
290     self.ax_mod.clear()
291     self.ax_mod.set_facecolor("#fad7ea")
292     self.ax_mod.set_title("S11 - Módulo")
293     self.ax_mod.set_ylabel("|S11| (dB)")
294     self.ax_mod.grid(True, alpha=0.3)
295     (self.linea_mod,) = self.ax_mod.plot([], [], color="deeppink",
296                                           marker=".", linestyle="None")
297
298     self.ax_fase.clear()
299     self.ax_fase.set_facecolor("#fad7ea")
300     self.ax_fase.set_title("S11 - Fase")
301     self.ax_fase.set_xlabel("Frecuencia de RF (MHz)")
302     self.ax_fase.set_ylabel("Fase (grados)")
303     self.ax_fase.grid(True, alpha=0.3)
304     (self.linea_fase,) = self.ax_fase.plot([], [], color="darkmagenta",
305                                           marker=".", linestyle="None")
306     self.fig_mf.tight_layout()
307
308 @staticmethod
309 def _nuevo_spin(valor, minimo, maximo, sufijo):
310     sp = QDoubleSpinBox()
311     sp.setDecimals(3)
312     sp.setRange(minimo, maximo)
313     sp.setValue(valor)
314     sp.setSuffix(sufijo)
315     return sp
316
317 @staticmethod
318 def _fila(etiqueta, widget):
319     w = QWidget()
320     h = QHBoxLayout(w)
321     h.setContentsMargins(0, 0, 0, 0)
322     h.addWidget(QLabel(etiqueta))
323     h.addWidget(widget)
324     return w
325
326 def _log(self, texto):
327     self.txt_log.appendPlainText(texto)
328
329 # ----- Conexion --
330
331 def _conectar_arduino(self):
332     puerto = self.ed_puerto.text().strip()
333     try:
334         baudrate = int(self.ed_baudrate.text().strip())
335     except ValueError:
336         QMessageBox.warning(self, "Baudrate invalido", "El baudrate debe ser
337                               ↪ un numero entero.")
338     return
339
340     try:

```

```

340         self.ser, linea = core.conectar_arduino(puerto, baudrate)
341     except Exception as e:
342         QMessageBox.critical(self, "Error de conexion", str(e))
343         self.lbl_estado_conexion.setText("Error de conexion")
344         return
345
346     self.lbl_estado_conexion.setText(f"Conectado ({puerto} @ {baudrate})")
347     if linea:
348         self._log(f"Arduino: {linea}")
349     self._log(f"Conectado a {puerto} @ {baudrate} baud.")
350
351     # ----- Utilidades -----
352
353     def _construir_frecuencias(self):
354         inicio = self.sp_f_inicio.value()
355         fin = self.sp_f_fin.value()
356         paso = self.sp_f_paso.value()
357         return core.generar_frecuencias(inicio, fin, paso)
358
359     def _construir_filtro(self):
360         return core.diseñar_filtro_pasabanda(
361             self.config["fs_audio"], self.config["f_centro_filtro"],
362             self.config["ancho_banda_filtro"], self.config["numtaps_filtro"],
363             self.config["duracion_captura_s"],
364         )
365
366     def _validar_listo_para_barrer(self):
367         if self.ser is None:
368             QMessageBox.warning(self, "Sin conexion", "Primero conecta el
369                 ↪ Arduino.")
370             return False
371         if self.hilo is not None and self.hilo.isRunning():
372             QMessageBox.warning(self, "Barrido en curso", "Ya hay un barrido
373                 ↪ corriendo.")
374             return False
375         return True
376
377     def _lanzar_barrido(self, frecuencias, on_terminado, on_abortado=None,
378         ↪ on_error=None):
379         coef_filtro = self._construir_filtro()
380         self.hilo = SweepWorker(self.ser, frecuencias, coef_filtro, self.config)
381         self.hilo.punto_medido.connect(self._on_punto_generico)
382         self.hilo.terminado.connect(on_terminado)
383         self.hilo.abortado.connect(on_abortado or self._on_abortado_generico)
384         self.hilo.error.connect(on_error or self._on_error_generico)
385         self.progreso.setValue(0)
386         self.btn_abortar.setEnabled(True)
387         self.hilo.start()
388
389     def _on_punto_generico(self, i, n, f_rf, s11, f_batido):
390         self.progreso.setMaximum(n)
391         self.progreso.setValue(i)

```

```

389         mod_db = 20 * np.log10(abs(s11) + 1e-12)
390         fase_deg = np.degrees(np.angle(s11))
391         self._log(f"[{i}]/[{n}] RF={f_rf:.3f} MHz batido={f_batido:.1f} Hz "
392                 f"|S11|={mod_db:.2f} dB fase={fase_deg:.1f} deg")
393
394     def _on_abortado_generico(self):
395         self.btn_abortar.setEnabled(False)
396         self._log("Barrido abortado por el usuario.")
397
398     def _on_error_generico(self, mensaje):
399         self.btn_abortar.setEnabled(False)
400         QMessageBox.critical(self, "Error durante el barrido", mensaje)
401         self._log(f"ERROR: {mensaje}")
402
403     def _abortar_barrido(self):
404         if self.hilo is not None and self.hilo.isRunning():
405             self.hilo.abortar()
406
407     # ----- Calibracion -----
408
409     ETIQUETAS_PASO_CAL = {
410         1: "Paso 1/3: conecte SHORT (cortocircuito) y presione 'Barrer paso"
411            ↪ actual'.",
412         2: "Paso 2/3: conecte OPEN (circuito abierto) y presione 'Barrer paso"
413            ↪ actual'.",
414         3: "Paso 3/3: conecte LOAD (50 ohm) y presione 'Barrer paso actual'.",
415     }
416
417     def _iniciar_calibracion(self):
418         if not self._validar_listo_para_barrer():
419             return
420         self.cal_paso = 1
421         self.cal_gm_cc = self.cal_gm_ca = self.cal_gm_50 = None
422         self.cal_disponible = False
423         self.btn_medir.setEnabled(False)
424         self.btn_cal_guardar.setEnabled(False)
425         self.btn_cal_paso.setEnabled(True)
426         self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[1])
427
428     def _barrer_paso_calibracion(self):
429         if not self._validar_listo_para_barrer():
430             return
431         frecuencias = self._construir_frecuencias()
432         self.btn_cal_paso.setEnabled(False)
433         self.btn_cal_iniciar.setEnabled(False)
434         self._lanzar_barrido(
435             frecuencias,
436             on_terminado=self._on_paso_calibracion_terminado,
437             on_abortado=self._on_calibracion_abortada,
438         )
439
440     def _on_calibracion_abortada(self):

```

```

439         self._on_abortado_generico()
440         self.btn_cal_paso.setEnabled(True)
441         self.btn_cal_iniciar.setEnabled(True)
442
443     def _on_paso_calibracion_terminado(self, freqs, gm):
444         self.btn_abortar.setEnabled(False)
445         self.btn_cal_iniciar.setEnabled(True)
446         self.cal_freqs = freqs
447
448         if self.cal_paso == 1:
449             self.cal_gm_cc = gm
450             self.cal_paso = 2
451             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[2])
452             self.btn_cal_paso.setEnabled(True)
453         elif self.cal_paso == 2:
454             self.cal_gm_ca = gm
455             self.cal_paso = 3
456             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[3])
457             self.btn_cal_paso.setEnabled(True)
458         elif self.cal_paso == 3:
459             self.cal_gm_50 = gm
460             self._finalizar_calculo_calibracion()
461
462     def _finalizar_calculo_calibracion(self):
463         n = len(self.cal_freqs)
464         if not (len(self.cal_gm_cc) == len(self.cal_gm_ca) ==
465             ↪ len(self.cal_gm_50) == n):
466             QMessageBox.warning(
467                 self, "Inconsistencia",
468                 "Los tres barridos de calibracion (SHORT/OPEN/LOAD) no tienen la
469                 ↪ "
470                 "misma cantidad de puntos. Revisa que no se haya abortado
471                 ↪ ninguno "
472                 "a mitad de camino."
473             )
474             self.cal_paso = 0
475             self.btn_cal_paso.setEnabled(False)
476             return
477
478         self.cal_e00, self.cal_e11, self.cal_delta_e = core.calculo_errores(
479             self.cal_gm_cc, self.cal_gm_ca, self.cal_gm_50, n
480         )
481         self.cal_disponible = True
482         self.cal_paso = 4
483         self.lbl_estado_cal.setText(f"Calibracion lista ({n} puntos).")
484         self.btn_cal_paso.setEnabled(False)
485         self.btn_cal_guardar.setEnabled(True)
486         self.btn_medir.setEnabled(True)
487         self._log("Calibracion SOL calculada correctamente.")
488
489     def _cargar_calibracion(self):
490         path, _ = QFileDialog.getOpenFileName(

```

```

488         self, "Cargar calibracion", "", "CSV (*.csv)"
489     )
490     if not path:
491         return
492     try:
493         freqs, e00, e11, delta_e = core.cargar_calibracion_csv(path)
494     except Exception as e:
495         QMessageBox.critical(self, "Error al cargar", str(e))
496         return
497
498     self.cal_freqs, self.cal_e00, self.cal_e11, self.cal_delta_e = freqs,
499     ↪ e00, e11, delta_e
500     self.cal_disponible = True
501     self.cal_paso = 4
502     self.lbl_estado_cal.setText(f"Calibracion cargada desde archivo
503     ↪ ({len(freqs)} puntos).")
504     self.btn_medir.setEnabled(True)
505     self.btn_cal_guardar.setEnabled(False)
506     self._log(f"Calibracion cargada de {path}")
507
508 def _guardar_calibracion(self):
509     if not self.cal_disponible:
510         return
511     path, _ = QFileDialog.getSaveFileName(
512         self, "Guardar calibracion", "calibracion.csv", "CSV (*.csv)"
513     )
514     if not path:
515         return
516     core.guardar_calibracion_csv(path, self.cal_freqs, self.cal_e00,
517     ↪ self.cal_e11, self.cal_delta_e)
518     self._log(f"Calibracion guardada en {path}")
519
520 # ----- Medicion DUT -----
521
522 def _iniciar_medicion_dut(self):
523     if not self._validar_listo_para_barrer():
524         return
525     if not self.cal_disponible:
526         QMessageBox.warning(self, "Sin calibracion", "Primero completa o
527         ↪ carga una calibracion.")
528         return
529
530     frecuencias = self._construir_frecuencias()
531
532     # Reiniciar buffers y graficos
533     self.buf_freq, self.buf_mod_db, self.buf_fase_deg, self.buf_gamma = [],
534     ↪ [], [], []
535     self._preparar_ejes_mod_fase()
536     self.canvas_mf.draw()
537     dibujar_fondo_smith(self.ax_smith)
538     (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
539     ↪ marker=".", linestyle="None")

```

```

534         self.canvas_smith.draw()
535
536         self.btn_medir.setEnabled(False)
537         self.btn_guardar_resultados.setEnabled(False)
538
539         self._lanzar_barrido(
540             frecuencias,
541             on_terminado=self._on_medicion_dut_terminada,
542             on_abortado=self._on_medicion_dut_abortada,
543         )
544         # Reconectamos punto_medido a un manejador especifico para DUT
545         # (ademas del log generico que ya se conecta en _lanzar_barrido)
546         self.hilo.punto_medido.connect(self._on_punto_dut)
547
548     def _on_medicion_dut_abortada(self):
549         self._on_abortado_generico()
550         self.btn_medir.setEnabled(True)
551
552     def _indice_calibracion_valido(self, i_1_based):
553         idx = i_1_based - 1
554         return self.cal_e00 is not None and 0 <= idx < len(self.cal_e00)
555
556     def _on_punto_dut(self, i, n, f_rf, s11_crudo, f_batido):
557         # Si hay calibracion punto-a-punto disponible, corregimos en vivo.
558         if self._indice_calibracion_valido(i):
559             idx = i - 1
560             gamma = core.correccion_punto(
561                 s11_crudo, self.cal_e00[idx], self.cal_e11[idx],
562                 ↪ self.cal_delta_e[idx]
563             )
564         else:
565             gamma = s11_crudo # fallback: se corrige todo al final
566
567         mod_db = 20 * np.log10(abs(gamma) + 1e-12)
568         fase_deg = np.degrees(np.angle(gamma))
569
570         self.buf_freq.append(f_rf)
571         self.buf_mod_db.append(mod_db)
572         self.buf_fase_deg.append(fase_deg)
573         self.buf_gamma.append(gamma)
574
575         self.linea_mod.set_data(self.buf_freq, self.buf_mod_db)
576         self.linea_fase.set_data(self.buf_freq, self.buf_fase_deg)
577         for ax in (self.ax_mod, self.ax_fase):
578             ax.relim()
579             ax.autoscale_view()
580         self.canvas_mf.draw_idle()
581
582         self.linea_smith.set_data([g.real for g in self.buf_gamma],
583                                   [g.imag for g in self.buf_gamma])
584         self.canvas_smith.draw_idle()

```

```

585 def _on_medicion_dut_terminada(self, freqs, gm_dut):
586     self.btn_abortar.setEnabled(False)
587     self.btn_medir.setEnabled(True)
588
589     if len(freqs) != len(self.cal_e00):
590         QMessageBox.warning(
591             self, "Advertencia de consistencia",
592             "La cantidad de puntos medidos no coincide con la calibracion "
593             "(revisa el rango de frecuencias). Se muestran los valores "
594             "corregidos en vivo, pero puede haber un corrimiento de indice."
595         )
596
597     n = min(len(freqs), len(self.cal_e00))
598     gamma_corregido = core.correccion(
599         gm_dut[:n], self.cal_e00[:n], self.cal_e11[:n],
600         ↪ self.cal_delta_e[:n], n
601     )
602     mod_db, fase_deg = core.modulo_fase(gamma_corregido)
603
604     self.ultima_medicion_freqs = freqs[:n]
605     self.ultima_medicion_gamma = gamma_corregido
606     self.ultima_medicion_mod_db = mod_db
607     self.ultima_medicion_fase_deg = fase_deg
608
609     self.btn_guardar_resultados.setEnabled(True)
610     self._log(f"Medicion de DUT finalizada: {n} puntos corregidos.")
611
612 def _guardar_resultados(self):
613     if self.ultima_medicion_freqs is None:
614         return
615     path, _ = QFileDialog.getSaveFileName(
616         self, "Guardar resultados", "resultado_S11.csv", "CSV (*.csv)"
617     )
618     if not path:
619         return
620     core.guardar_resultados_csv(
621         path, self.ultima_medicion_freqs,
622         self.ultima_medicion_mod_db, self.ultima_medicion_fase_deg,
623     )
624     self._log(f"Resultados guardados en {path}")
625
626 # ----- Cierre -----
627
628 def closeEvent(self, event):
629     if self.hilo is not None and self.hilo.isRunning():
630         self.hilo.abortar()
631         self.hilo.wait(2000)
632     if self.ser is not None:
633         try:
634             self.ser.close()
635         except Exception:
636             pass

```

```
636         event.accept()
637
638
639 def main():
640     app = QApplication(sys.argv)
641     app.setStyleSheet("""
642         QMainWindow, QWidget, QGroupBox {background-color: #fad7ea; }""")
643     ventana = MainWindow()
644     ventana.show()
645     sys.exit(app.exec())
646
647
648 if __name__ == "__main__":
649     main()
```

Apéndice

UNMDP-FI

Carrera: Ingeniería Electrónica

Seminario-Taller para el Diseño de Soluciones Tecnológicas

Victoria Carla Torres

***Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia***



Versión 2.0

Victoria Carla Torres

Contenido

ACERCA DEL EQUIPO	3
INTEGRANTES	3
DIRECTOR/A	3
CO-DIRECTOR/A	3
ACERCA DEL TRABAJO FINAL	3
DESCRIPCIÓN DE LA NECESIDAD O IDEA	3
BENEFICIARIOS / CLIENTES / USUARIOS	3
PRODUCTO/S O ENTREGABLE/S RESULTANTE/S	3
ESQUEMA PRELIMINAR DE LA SOLUCIÓN	3
GRADO DE AVANCE DEL TRABAJO FINAL	3

**Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia**

Acerca del Equipo**Integrantes**

Victoria Carla Torres

Estudiante y becaria en investigación, con una beca de Estudiante Avanzado de la UNMDP otorgada en 2025 y renovada para 2026. Ayudante estudiante en Física 2/B desde 2023 hasta la actualidad. Bachiller en Economía y Administración.

Director/a

Avalos Ribas, Ramiro. Jefe de Trabajos Prácticos, exclusiva.

Co-Director/a

Maximiliano Gluzman, Brian. Ayudante graduado, exclusiva.

Acerca del Trabajo Final**Descripción de la necesidad o idea**

Actualmente, el monitoreo de los niveles de glucemia requiere que los pacientes se sometan a métodos invasivos (punciones) que resultan incómodos y dolorosos. Por esta razón, existe una necesidad en el ámbito de la ingeniería de buscar la optimización de un sistema de medición no invasiva de glucemia.

El método se basa en medir de forma indirecta cómo varían las propiedades electromagnéticas de la sangre al ser atravesadas por señales de radiofrecuencia. Ya se han desarrollado en el Grupo de Investigación sistemas con este propósito, basados en un Analizador Vectorial de Redes (VNA) que complementan el sensor desarrollado. Se busca optimizar, compactar y mejorar el rendimiento del VNA actual.

Partes interesadas (stakeholders)

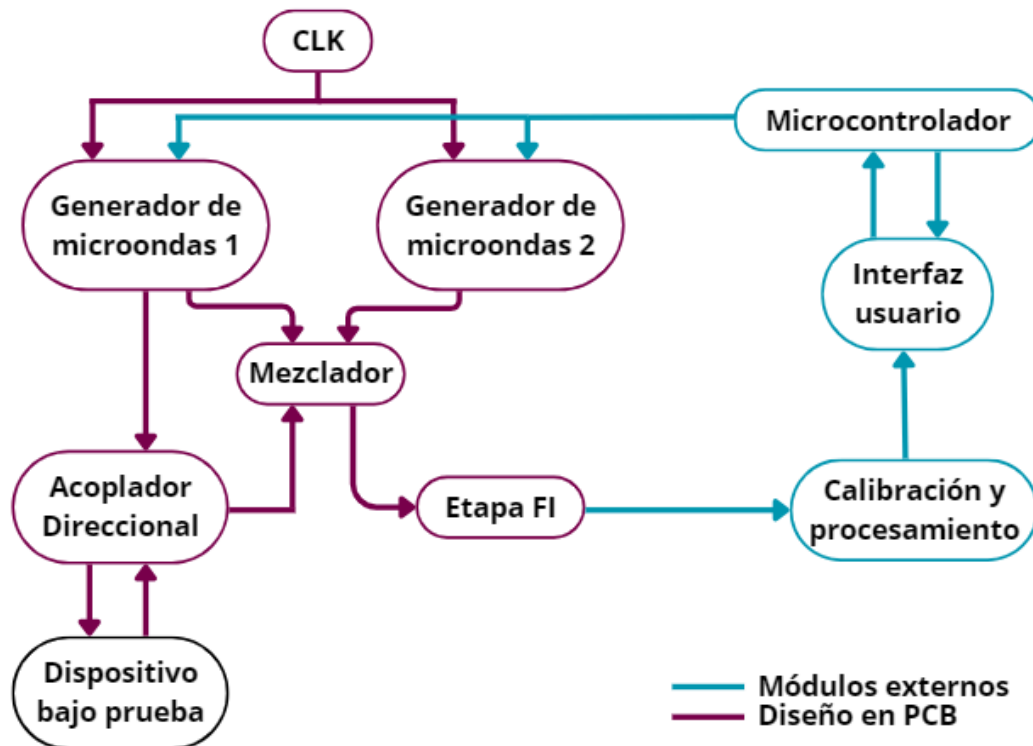
Solicita el desarrollo el Grupo de Investigación del Laboratorio de Comunicaciones. Ellos serán los receptores de la solución propuesta, y por lo tanto los usuarios serán ingenieros electrónicos, o estudiantes bajo supervisión.

Producto/s o entregable/s resultante/s

ϵ - μ VNA: placa de circuito impreso (PCB) que integra los bloques de reloj (CLK), generadores de microondas, acoplador direccional, mezclador y etapa de FI. Con conectores SMA, USB-B y conector de audio analógico, para medición, alimentación y conexión respectivamente.

**Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia**

Esquema preliminar de la solución



Grado de avance del Trabajo Final

Hasta el momento se ha desarrollado la placa con todos sus bloques con el software Altium Designer, y está en proceso de fabricación.

Las actividades pendientes son:

- Ensamblado;
- Implementación del sistema de adquisición, procesamiento y visualización de datos;
- Mediciones para determinar los resultados, funcionamiento, y si se deben hacer ajustes o modificaciones;
- Escribir el informe final.

UNMDP-FI

Carrera: Ingeniería Electrónica

Plan de Trabajo

Victoria Carla Torres

***Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia***



*14/05/2026
Versión 1.0*

Introducción

Se presenta la planificación de tareas, objetivos, cronograma de actividades e identificación de riesgos del desarrollo del trabajo final de grado “Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia”

Tareas a realizar

Las tareas a desarrollar para poder finalizar el proyecto son:

- Lectura de bibliografía.
- Diseño de la arquitectura del sistema.
- Análisis de los posibles componentes a utilizar a modo de: oscilador local, mezclador, generadores de RF, acoplador direccional.
- Elección y caracterización de los componentes.
- Diseño y ruteo en PCB.
- Ensamblado de las partes del proyecto.
- Ensayos y pruebas para verificar el funcionamiento.
- Redacción de entregables e informe final.

Cronograma de actividades

El cronograma se presenta en la figura 1 mediante un diagrama de Gantt, donde se detallan las tareas y el tiempo aproximado que se destina a resolverlas. Las figuras con forma de diamante corresponden a hitos del proyecto, explicados en la próxima sección de este documento.

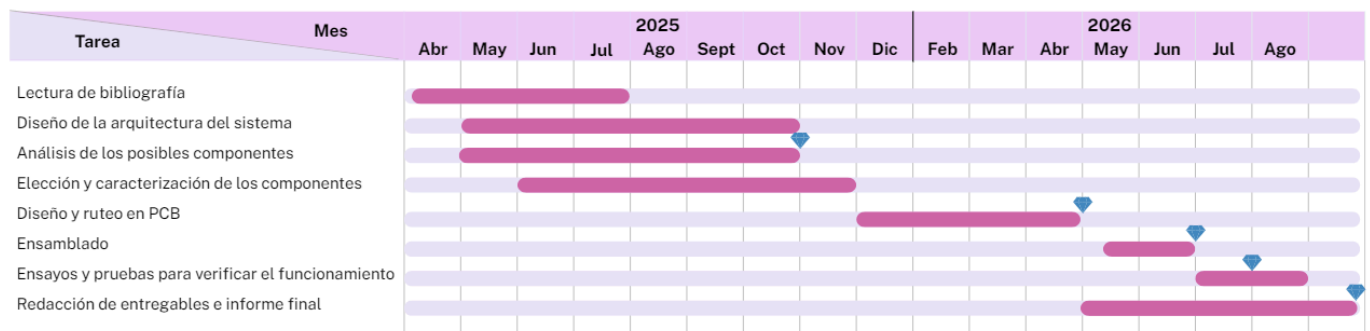


Figura 1: diagrama de Gantt

Hitos del proyecto

Los eventos importantes para marcar el avance del proyecto son:

- Aprobación por parte de los receptores de la arquitectura final y componentes a utilizar.
- Aprobación del diseño de PCB y ruteo.
- Ensamblado de la placa terminado.
- Pruebas exitosas:
 - Que se pueda generar un barrido de señales.

- Que el acoplador direccional presente una impedancia de 50Ω , y los niveles deseados de acoplamiento y directividad.
- Que el microcontrolador sea capaz de pasar los datos al generador correctamente.
- Que la salida del mezclador tenga la señal de FI buscada.
- Aprobación por parte de los directores del informe final.

Asignación de responsables y recursos

En la tabla 1 se observa la distribución de tareas generales, con sus respectivos encargados y medios para cumplirlas.

Tarea	Responsable	Recursos
Simulaciones	Estudiante y profesor	Software de simulación (CST Studio, QucsStudio)
Elección de los componentes	Estudiante y profesor	Hojas de datos, bibliografía
Diseño del PCB	Estudiante	Software Altium Designer Pro, licencia estudiantil.
Ensamblado	Estudiante	Instrumental de laboratorio
Ensayos y pruebas	Estudiante y profesor (supervisión)	Instrumental de laboratorio
Redacción de entregables	Estudiante	Sistemas de composición de texto (Google Docs, LaTeX)

Tabla 1: asignación de responsables y recursos

Identificación de riesgos

En la tabla 2 se muestran los posibles riesgos para el cumplimiento del proyecto, junto con la probabilidad de ocurrencia, nivel de impacto en el proyecto, qué consecuencia tendría y qué se debe hacer en caso de que ocurra.

Riesgo	Probabilidad de ocurrencia	Impacto	Consecuencia	Acción reparadora
Inicio trabajo de tiempo completo	Bajo	Alto	Retraso en el cronograma	Redistribución de tiempos
Viaje al exterior por beca de intercambio	Medio bajo	Alto	Retraso en el cronograma	Redistribución de tiempos
Fallas de componentes	Medio	Medio	Retraso en el	Reemplazar

durante las pruebas			cronograma	componentes
Fallas en el diseño del PCB	Medio	Alto	Retraso en el cronograma	Rediseñar
Demora en la impresión del PCB	Medio alto	Medio	Retraso en el cronograma	Búsqueda de nuevos proveedores

Tabla 2: identificación de riesgos

Seguimiento del proyecto

El seguimiento del proyecto se realizará presencialmente en el lugar de trabajo de estudiante y directores, recurriendo a vías digitales (email, videoconferencias) ante cualquier eventualidad que condicione la presencialidad.

UNMDP-FI

Carrera: Ingeniería Electrónica

Especificación de Requerimientos

Victoria Carla Torres

***Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia***



Indice

1. Introducción.....	3
1.1 Alcance del proyecto.....	3
1. 2 Definiciones, Abreviaturas y Siglas.....	3
2. Descripción General.....	3
2.1 Perspectiva del Producto.....	3
2.1 Funciones del Producto.....	4
2.3 Características del Usuario.....	4
2.4 Datos a Asumir y Dependencias.....	4
3. Requerimientos Específicos.....	5

1. Introducción

Este documento tiene como propósito definir y describir los requerimientos del proyecto final “Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia” correspondiente a la carrera de Ingeniería Electrónica.

Un Analizador Vectorial de Redes (*Vector Network Analyzer* - VNA) es un instrumento de medición de radiofrecuencia (RF) diseñado para caracterizar el comportamiento de un dispositivo bajo prueba (*Device Under Test* - DUT). Su funcionamiento se basa en inyectar una señal de estímulo y medir su transmisión y reflexión, para extraer los parámetros “S” (*scattering*).

1.1 Alcance del proyecto

El proyecto propone el diseño, desarrollo, calibración y ensayo de un VNA que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para las necesidades de la aplicación. Para lograrlo se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en RF. Para realizar las mediciones se utiliza instrumental de laboratorio provisto por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

1.2 Definiciones, Abreviaturas y Siglas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
PCB	Printed Circuit Board
RF	Radio Frecuencia
LAC	Laboratorio de Comunicaciones
UNMDP	Universidad Nacional de Mar del Plata
SMD	Surface - Mount Device
SMA	SubMiniature version A
FI	Frecuencia Intermedia
ROE	Razón de Onda Estacionaria
DUT	Device Under Test
UFH	Ultra High Frequency

2. Descripción General

2.1 Perspectiva del Producto

El VNA es un subsistema que forma parte del proyecto de un Grupo de Investigación en el LAC para desarrollar un medidor no invasivo de glucemia, por lo tanto está pensado para que actúe en conjunto con un sensor que actúa de DUT cuando se introduce en él un dedo índice. El principio en el cual se basa el desarrollo es que las propiedades dieléctricas de la sangre varían proporcionalmente a los niveles de glucosa presentes en ella.

La conexión entre estos dos componentes se realiza mediante conectores SMA.

La interfaz de usuario consiste en un programa ejecutable de PC desde el cual se eligen los parámetros y se visualizan los resultados. Se utiliza además la placa de sonido de la PC para adquisición y tratamiento de datos.

2.1 Funciones del Producto

El sistema debe ser capaz de generar un barrido de señales de frecuencias que abarquen desde los 1,8 GHz hasta los 2,2 GHz. Estas se deben inyectar en el DUT con el fin de medir la onda reflejada. Luego, tanto esta señal como una muestra de la señal inducida deben pasar por un proceso de heterodinación, para trasladarlas a una frecuencia intermedia (FI) que permita adquirirlas y digitalizarlas por una etapa de adquisición de audio. En la figura 1 se puede observar un diagrama general del sistema completo.

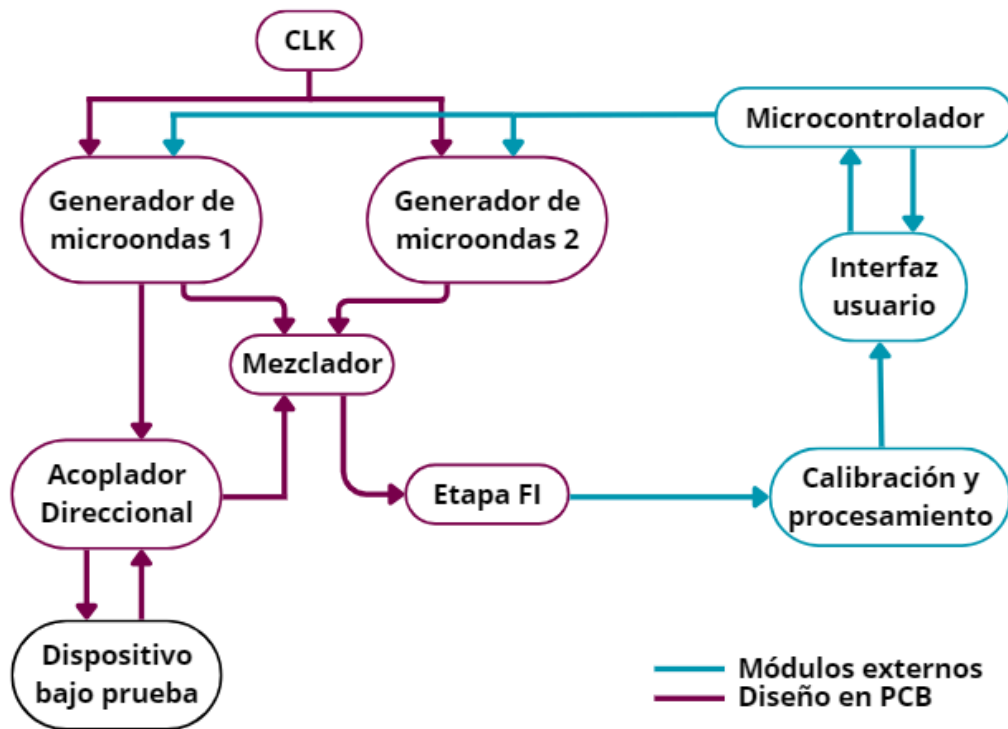


Figura 1: diagrama de bloques VNA

2.3 Características del Usuario

El Grupo de Investigación del LAC es quien solicita el desarrollo. Por esta razón, el equipo se utilizará bajo su supervisión, y sus posibles usuarios son:

- Ingenieros, técnicos y/o radioaficionados;
- Estudiantes de ingeniería.

2.4 Datos a Asumir y Dependencias

Se debe considerar a la hora de diseñar la arquitectura del sistema:

- Que la PC a utilizar debe poseer una placa de sonido con entrada de línea de audio analógica que permita la adquisición, cuya resolución limita rango dinámico y sensibilidad;
- Que se debe hacer una calibración para compensar errores sistemáticos del sistema, como los producidos por pérdidas de inserción en materiales conductores y conectores, desfasajes ocasionados por diferencias en longitudes eléctricos o retardos en los circuitos integrados, así como reflexiones generadas por desadaptación de impedancias;
- Que se deben considerar los comportamientos propios del trabajo en alta frecuencia;
- Que el instrumental disponible en el laboratorio permite medir señales de hasta 3GHz;
- Que los microcontroladores aceptan determinados lenguajes para su programación.

3. Requerimientos Específicos

3.1 Requerimientos funcionales

RF-01 [Ajuste del rango de medición]: el usuario debe poder definir el rango de frecuencias en el que va a realizar la medición, dentro del rango permitido, por medio de la interfaz de usuario.

RF-02 [Escritura de registros]: el microcontrolador debe recibir la información del barrido y calcular los registros que debe mandar a los generadores.

RF-03 [Barrido en frecuencia]: los bloques generadores deben recibir la información y comenzar el barrido de frecuencias.

RF-04 [Separación de onda incidente]: una parte de la potencia de la onda que es generada debe desviarse hacia el mezclador para tener una muestra de la onda incidente.

RF-05 [Heterodinación de señales]: el mezclador debe ser capaz de tomar las señales incidente, reflejada, y la señal externa que actúa como oscilador local y devolver dos señales diferenciales en FI.

RF-06 [Adquisición de audio]: la PC debe tomar las señales y digitalizarlas, devolviendo dos vectores con muestras.

RF-07 [Procesamiento de datos y muestra de resultados]: se debe poder procesar los datos y mostrarlos en un gráfico al usuario.

RF-08 [Rutina de Calibración]: el usuario debe poder realizar la calibración del dispositivo utilizando cargas de impedancias conocidas.

3.2 Requerimientos de performance

RP-01 [Rango de operación]: 1800 a 2200 MHz

RP-02 [Impedancia característica de líneas de transmisión]: 50 Ω .

RP-03 [Resolución del barrido]: los generadores deben permitir una resolución mínima de 1kHz.

RP-04 [Directividad mínima aceptable del acoplador direccional]: 15dB en la banda de interés

RP-05 [ROE mínima distinguible]: menor o igual a 1,5.

RP-06 [Frecuencia intermedia]: 1 a 20 kHz.

3.3 Pautas de diseño y Restricciones

El diseño del sistema está condicionado fuertemente por la naturaleza de las señales de RF. Por esta razón surgen pautas como la elección de utilizar componentes SMD, placas cuyo sustrato tiene ciertas propiedades dieléctricas, el control de impedancias en el diseño de líneas de transmisión, y la utilización de técnicas como el via fencing.

Otras pautas a tener en cuenta son las definidas por los estándares de fabricación de los proveedores, sus capacidades y especificaciones técnicas, que también limitan la elección de materiales y técnicas a utilizar por medio de stackups predefinidos, tolerancias, limitaciones de ancho de pistas y espaciados. A su vez la selección de componentes depende de la disponibilidad y posibilidad de fabricación, bajo el criterio de optimizar el diseño de forma funcional, económica y espacial. En este último aspecto, se debe llegar a una placa de un tamaño menor a $18 \times 17.5 \text{ cm}^2$.

En cuanto a las restricciones en el diseño de la arquitectura, se debe utilizar un oscilador a cristal de 10MHz, dando además la opción de usar un generador de señales de forma auxiliar. Se deben emplear generadores ADF4351 y tensiones de alimentación entre 3,3 y 5V.

Por último se deben considerar las características de los instrumentos de medición disponibles, que limitan la posibilidad de verificar el funcionamiento del sistema en ciertas señales internas a los integrados o en ciertas frecuencias, amplitudes de señales y potencias. Por lo tanto, se deben diseñar puntos de prueba accesibles que garanticen la integridad de las mediciones.

3.4 Atributos

- Confiabilidad: antes de realizar una medición, se debe realizar una calibración con distintas cargas de valores conocidos (típicamente circuito abierto, cortocircuito y 50Ω) a las frecuencias de operación para descartar errores sistemáticos.
- Mantenibilidad: se puede modificar y actualizar el firmware del microcontrolador y la interfaz de usuario, además de reemplazar componentes en caso de fallas, por otros semejantes.

3.5 Requerimientos de interface externa

3.5.1 Interfaces de usuario

El usuario debe poder acceder a una interfaz gráfica que le permita:

- Ajustar parámetros de medición
- Visualizar los resultados

3.5.2 Interfaces de hardware

Las interfaces de hardware presentes en el proyecto son:

- Entre el dispositivo bajo prueba (DUT) y VNA: conector SMA
- Entre la placa de sonido de la PC y la salida del VNA: conector de audio analógico
- Alimentación: USB-B
- Conectores SMA para mediciones de RF auxiliares;
- Conector SMA para entrada de referencia de RF externa.

UNMDP-FI

Carrera: Ingeniería Electrónica

***Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia***

Especificación Funcional

Solución



Versión 1.0

Especificación funcional

Fecha	Versión	Descripción	Autor/a
27/05/2026	1.0	Versión inicial	Victoria Carla Torres

Índice

1. Introducción.....	4
1.1 Propósito del documento.....	4
1.2 Alcance del proyecto.....	4
1.3 Personal involucrado.....	4
1.4 Definiciones, acrónimos y abreviaturas.....	5
1.5 Referencias.....	5
2. Descripción del dispositivo.....	5
3. Especificaciones funcionales.....	6
3.1 RF-01: Ajuste del rango de medición.....	6
3.2 RF-02 Escritura de registros.....	6
3.3 RF-03 Barrido en frecuencia.....	6
3.4 RF-04 Separación de onda incidente.....	6
3.5 RF-05 Heterodinación de señales.....	7
3.6 RF-06 Adquisición de audio.....	7
3.7 RF-07 Procesamiento de datos y muestra de resultados.....	7
3.8 RF-08 Rutina de Calibración.....	7
3.9 RF-09: Alimentación.....	8
4. Requerimientos no funcionales.....	8
4.1 RNF-01: Tamaño de placa de PCB.....	8
4.2 RNF-03: Rango de operación.....	8
4.3 RNF-04: Impedancia característica.....	8
4.4 RNF-05: Resolución del barrido.....	8
4.5 RNF-06: Directividad mínima aceptable del acoplador direccional.....	8
4.6 RNF-07: ROE mínima distinguible.....	8

Especificación funcional

1. Introducción

Este documento corresponde a la Especificación Funcional para la solución “Analizador Vectorial de Redes para medición no invasiva de glucemia”. Esta especificación se ha estructurado basándose en la información mencionada en el documento de Especificación de Requerimientos (ER) “[ER_Torres-v1](#)”.

1.1 Propósito del documento

El presente documento tiene como propósito proveer información detallada de cómo debe funcionar el sistema, cuáles son sus comportamientos deseados y cómo se debe construir, con base en los requerimientos anteriormente definidos en la ER.

Se dirige al desarrollador del proyecto (quien lo va a construir) y a los solicitantes del proyecto, directores y evaluadores.

1.2 Alcance del proyecto

El proyecto propone el diseño, desarrollo, calibración y ensayo de un Analizador Vectorial de Redes (VNA) que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para medición no invasiva de glucemia. Para lograrlo se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en radiofrecuencia. Para realizar las mediciones se utiliza instrumental de laboratorio provisto por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

1.3 Personal involucrado

Nombre	Victoria Carla Torres
Rol	Estudiante
Categoría Profesional	Bachiller
Responsabilidad	Desarrollo, diseño, escritura de documentos
Información de contacto	victoriatorres@mdp.edu.ar

Nombre	Ramiro Avalos Ribas
Rol	Director
Categoría Profesional	Doctor Ingeniero
Responsabilidad	Supervisión, aprobación y revisión del desarrollo
Información de contacto	avalosribas@fi.mdp.edu.ar

Nombre	Brian Maximiliano Gluzman
Rol	Codirector
Categoría Profesional	Ingeniero
Responsabilidad	Supervisión, aprobación y revisión del desarrollo
Información de contacto	briangluzman@mdp.edu.ar

1.4 Definiciones, acrónimos y abreviaturas

Nombre	Descripción
RF	Requerimiento Funcional
RNF	Requerimiento No Funcional
VNA	Analizador Vectorial de Redes
PC	Personal Computer
PCB	Printed Circuit Board
LAC	Laboratorio de Comunicaciones
UNMDP	Universidad Nacional de Mar del Plata
SMD	Surface - Mount Device
SMA	SubMiniature version A
FI	Frecuencia Intermedia
ROE	Razón de Onda Estacionaria
DUT	Device Under Test
UFH	Ultra High Frequency

1.5 Referencias

[1] D. Rytting, "Network analyzer error models and calibration methods," White paper, Infineon Technologies AG, Santa Clara, CA, USA, Sep. 1998.

2. Descripción del dispositivo

En la Figura 1 se puede observar un diagrama en bloques del sistema propuesto. Se compone de tres subsistemas principales:

- ϵ - μ VNA: placa de PCB (Printed Circuit Board) donde se deben realizar la generación de señales, la medición y la heterodinación para obtener dos señales de una FI (Frecuencia Intermedia) en la banda de audio, una de las cuales contiene información del dispositivo bajo prueba (DUT - Device Under Test)

- PC: allí se deben realizar la adquisición y el procesamiento de las señales, además de alojar la interfaz de usuario. El ϵ - μ VNA se debe comunicar con ella por medio de un conector de audio analógico. A su vez, ésta se debe comunicar con el microcontrolador para indicar los parámetros de configuración de barrido.

- Microcontrolador: se debe encargar de programar los registros de los generadores de radiofrecuencia a partir de las frecuencias elegidas por el usuario en la PC.

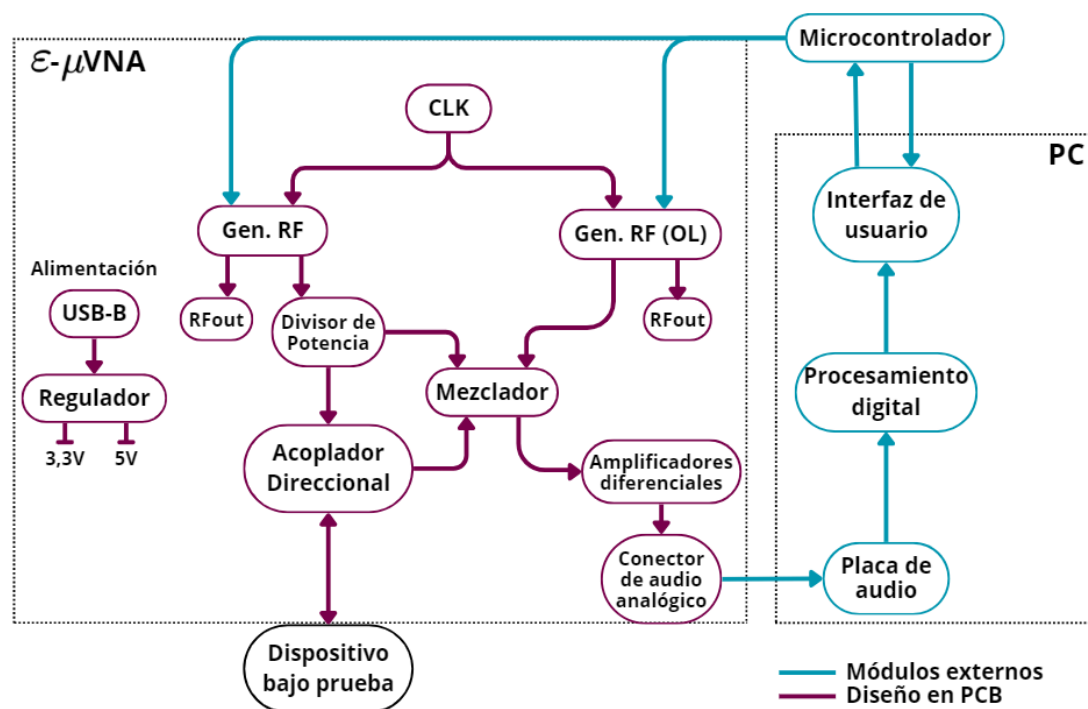


Figura 1: diagrama en bloques del sistema

3. Especificaciones funcionales

3.1 RF-01: Ajuste del rango de medición

El usuario debe poder definir el rango de frecuencias en el que va a realizar la medición, dentro del rango permitido, por medio de la interfaz de usuario.

3.2 RF-02 Escritura de registros

El microcontrolador debe recibir la información del barrido y calcular los registros que debe mandar a los generadores.

3.3 RF-03 Barrido en frecuencia

Los bloques generadores deben guardar los registros y comenzar el barrido de frecuencias definido por el usuario.

3.4 RF-04 Separación de onda incidente

Una parte de la potencia de la onda que es generada debe desviarse hacia el mezclador para tener una muestra de la onda incidente (inc). La otra parte debe ingresar al acoplador direccional para incidir en el DUT, lo que produce una onda reflejada, de menor amplitud, que depende de la impedancia de entrada. Esta onda (ref) se debe dirigir al mezclador como muestra la Figura 2.

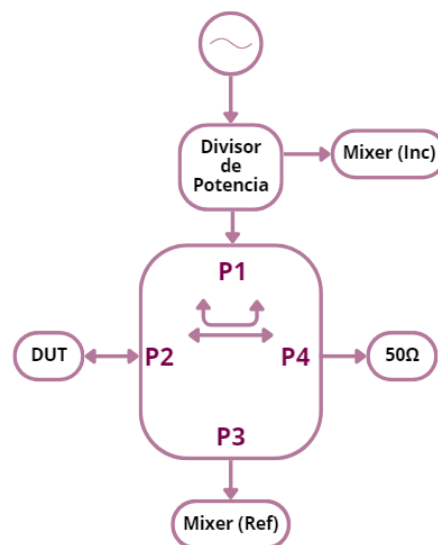


Figura 2: diagrama de la separación de ondas

3.5 RF-05 Heterodinación de señales

El mezclador debe ser capaz de recibir las señales incidente, reflejada, y la señal externa que actúa como oscilador local y devolver dos señales diferenciales en FI, como se puede visualizar en la Figura 3.

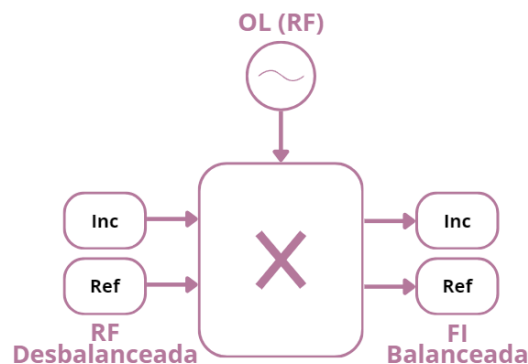


Figura 3: heterodinación de señales

3.6 RF-06 Adquisición de audio

La PC debe adquirir las señales y digitalizarlas, generando dos vectores con sus muestras.

3.7 RF-07 Procesamiento de datos y muestra de resultados

Se debe aplicar un procesamiento a los datos para calcular los parámetros de amplitud y fase de cada señal y generar el parámetro S correspondiente, para todo el barrido de frecuencias. Luego se los debe enviar hacia un bloque de almacenamiento para mostrarse en un gráfico al usuario o bien guardarse para calibración.

3.8 RF-08 Rutina de Calibración

El usuario debe poder realizar la calibración del dispositivo utilizando el método SOL (Short-Open-Load), cuyo modelo es el de la Figura 4 [1].

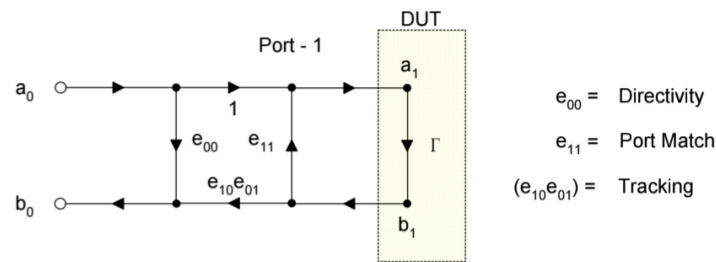


Figura 4: gráfico de flujo de calibración de un puerto y tres términos.

3.9 RF-09: Alimentación

Las tensiones de alimentación deben encontrarse entre 3,3 y 5V.

4. Requerimientos no funcionales

4.1 RNF-01: Tamaño de placa de PCB

El tamaño final de la placa debe ser menor a 18x17.5 cm².

4.2 RNF-02: Rango de operación

Se debe poder operar entre 1800 y 2200 MHz

4.3 RNF-03: Impedancia característica

Las líneas de transmisión deben tener una impedancia característica de 50Ω

4.4 RNF-04: Resolución del barrido

Los generadores de radiofrecuencia deben permitir una resolución mínima de 1kHz.

4.5 RNF-05: Directividad mínima aceptable del acoplador direccional

La directividad mínima que debe presentar el acoplador direccional debe ser de 15dB en la banda de interés.

4.6 RNF-06: ROE mínima distinguible

La ROE mínima distinguible debe ser menor o igual a 1,5.

UNMDP-FI

Carrera: Ingeniería Electrónica

***Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia***

Especificación Técnica



Versión 2.0

Especificación técnica

Fecha	Versión	Descripción	Autor/a
27/08/2026	1.0	Versión inicial	Victoria Carla Torres
21/09/2026	2.0	Corrección	Victoria Carla Torres

Índice

1. Introducción	4
1.1 Propósito del documento	4
1.2 Alcance del proyecto	4
1.3 Definiciones, acrónimos y abreviaturas	4
2. Hardware	5
2.1. ϵ - μ VNA	5
2.1.1 CLK	5
2.1.2 Generadores de RF	5
2.1.3 Mezclador	6
2.1.4 Acoplador direccional	7
2.1.5 Amplificadores diferenciales	7
2.1.6 Alimentación	7
2.1.7 Conectores	7
2.2 Diseño PCB	7
3. Firmware	9
4. Software	10
5. Pruebas técnicas	11
6. Referencias	11

1. Introducción

Este documento corresponde a la Especificación Técnica para la solución “Analizador Vectorial de Redes para medición no invasiva de glucemia”. Esta especificación se ha estructurado basándose en la información mencionada en el documento de Especificación Funcional (EF) “[EF_Torres-v1](#)”.

1.1 Propósito del documento

El presente documento provee información detallada de cómo debe funcionar el sistema, cuáles son sus comportamientos deseados y cómo se debe construir, con base en los requerimientos anteriormente definidos en la EF.

Se dirige al desarrollador del proyecto (quien lo va a construir) y a los solicitantes del proyecto, directores y evaluadores.

1.2 Alcance del proyecto

El proyecto propone el diseño, desarrollo, calibración y ensayo de un Analizador Vectorial de Redes (VNA) que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para medición no invasiva de glucemia. Se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en radiofrecuencia. Para realizar las mediciones se utiliza instrumental de laboratorio provisto por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

1.3 Definiciones, acrónimos y abreviaturas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
UHF	Ultra High Frequency
SMD	Surface - Mount Device
LAC	Laboratorio de Comunicaciones
UNMDP	Universidad Nacional de Mar del Plata
PCB	Printed Circuit Board
IC	Integrated Circuit
DUT	Device Under Test
SMA	SubMiniature version A
FI	Frecuencia Intermedia
ROE	Razón de Onda Estacionaria
PC	Personal Computer

2. Hardware

El sistema general está compuesto por tres módulos principales, que se pueden observar en la figura 1. Se encargan, en términos generales: ϵ - μ VNA, de la generación de radiofrecuencias (RF), mediciones y de trasladarlas a una Frecuencia Intermedia (FI) para poder luego adquirirlas; PC, realiza la adquisición, procesamiento de señales y muestra de resultados; y microcontrolador que realiza los cálculos para luego programar los registros de los generadores de RF a partir de las frecuencias elegidas por el usuario.

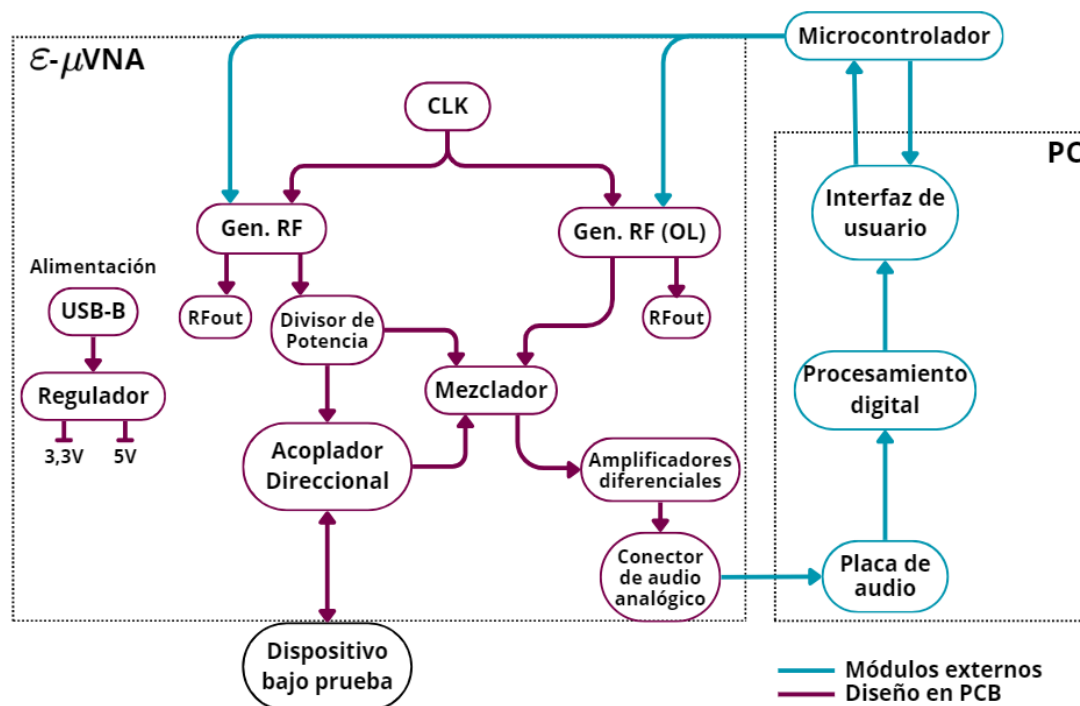


Figura 1: diagrama en bloques del sistema

2.1. ϵ - μ VNA

La construcción del dispositivo ϵ - μ VNA consiste en una placa de circuito impreso (PCB - Printed Circuit Board). Los integrados a utilizar y sus funciones, según los bloques presentados en la figura 1, son:

2.1.1 CLK

Oscilador a cristal ECS-2200BX-100 [1]. Se utiliza como referencia para los generadores de RF, junto con un buffer 74LVC2G07W6-7 [2] que permite bifurcar la señal para que ingrese a los generadores en forma independiente.

2.1.2 Generadores de RF

Basados en el IC ADF4351[3], sintetizador de banda ancha con VCO integrado, generan señales de frecuencia programable por interfaz serie desde 35 a 4400 MHz con potencia de salida programable, y tensión de alimentación de 3 a 3,6 V. Uno se utiliza como señal incidente en el acoplador direccional, mientras que el otro hace las veces de oscilador local para el proceso de heterodinaje junto con el mezclador. El esquemático general es el que se ve en la figura 2. Allí se observa una salida auxiliar para medición de las frecuencias generadas, un led con postes para verificar el enganche, capacitores de desacople en la alimentación y para estabilidad en pines requeridos por la hoja de datos (como TEMP, Vcom, y Vref). Entre los pines de Vtune y CPout

Especificación técnica

está el filtro externo del lazo de control. Fue diseñado en la aplicación de Analog Devices ADISIMPLL 5.9 [4], para cumplir con un margen de fase de 49° y ancho de banda de 9,9 kHz.

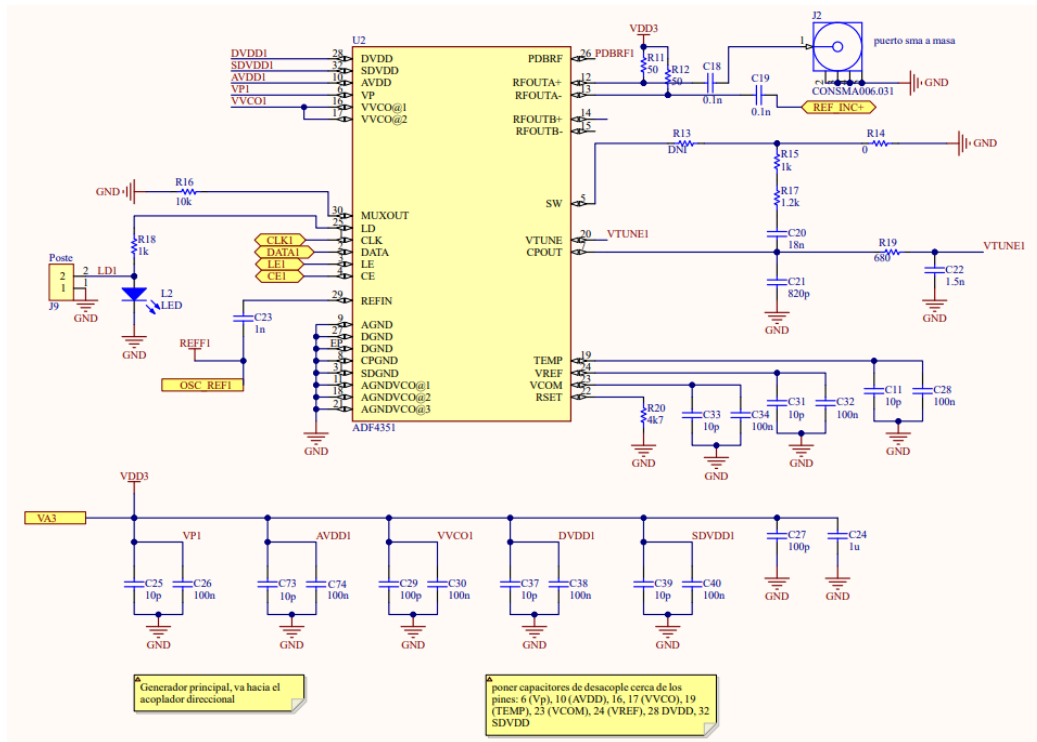


Figura 2: esquemático ADF4351.

2.1.3 Mezclador

Consiste en el IC ADL5802 [5], mezclador activo de dos canales que funciona de 100 MHz a 6 GHz con una ganancia de conversión de potencia de 1,6dB, y se alimenta con 5V. Tiene salidas diferenciales de 200Ω de colector abierto, por lo que se colocan resistencias de pull-up. Se agregan filtros RC simples de frecuencia de corte para atenuar la frecuencia suma. Se puede observar el esquemático en la figura 3.

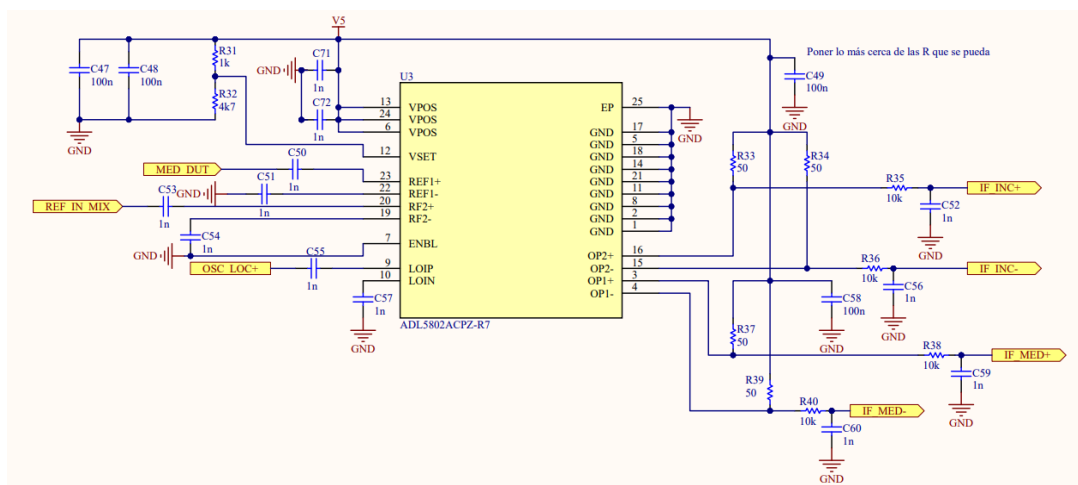


Figura 3: esquemático ADL5802.

Especificación técnica

2.1.4 Acoplador direccional

Basado en un puente de Wheatstone, red de resistencias que permite la comparación de impedancias con un ancho de banda teórico infinito. Como se observa en la figura 4, una parte de la señal que proviene del generador de RF se desvía hacia el mezclador, mientras que otra excita el puente resistivo. Este posee dos transformadores de RF TC1-1-13M+ [6] en cascada, que permiten medir la señal reflejada de manera desbalanceada y filtrar el ruido de modo común que provocaría utilizar solo uno. Las resistencias de 68Ω logran que la impedancia vista desde el puerto de salida (J1) sea 50Ω .

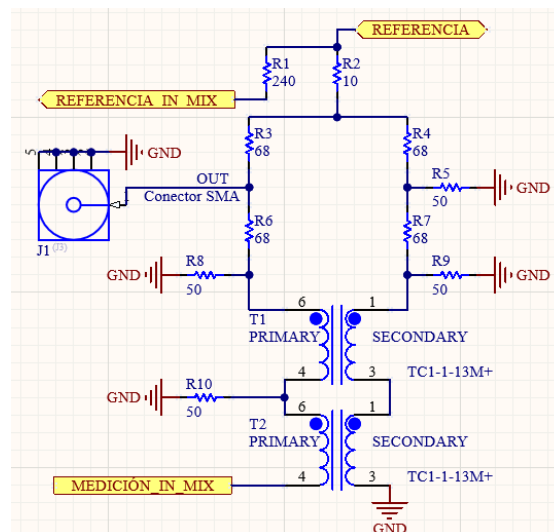


Figura 4: puente resistivo

2.1.5 Amplificadores diferenciales

Amplificadores operacionales MCP6023 [7]. Son restadores que toman la señal en FI proveniente del mezclador y la acondicionan para la entrada de línea de audio analógica de la PC, permiten obtener una señal desbalanceada y amplificada.

2.1.6 Alimentación

La alimentación general de 5V puede provenir de un USB-B o de una fuente de laboratorio. Un regulador de tensión UA78M33CDCYR [8] provee los 3,3V necesarios para alimentar los generadores de RF y el oscilador.

2.1.7 Conectores

Los conectores a utilizar son: SMA para mediciones de RF y entrada de referencia auxiliar; conector de audio analógico para entrada a la PC; USB-B para alimentación y tiras de pines junto con cables puente para escribir los registros de los ADF4351, entrada de alimentación auxiliar y medición de enganche.

2.2 Diseño PCB

La implementación en PCB es diseñada en el software Altium Designer, del que se dispone una licencia estudiantil gracias al programa Altium Student Lab. El diseño posee cuatro capas, debido a la dificultad para rutear pistas en dos capas. Un ejemplo es el caso de los pines Vtune y CPout de los ADF4351, que requieren atravesar caminos de RF por lo que se interrumpiría el camino de la corriente de retorno. Además, las pistas son susceptibles a interferencias electromagnéticas. Otro ejemplo es poder llevar alimentación a regiones rodeadas

de pistas de RF, por los mismos motivos.

Los componentes se sueldan en la capa 1. La capa 2 corresponde a un plano de masa continuo, la capa 3 se destina a alimentación. La última capa se usa para ruteo.

Para las trazas de RF se utilizaron líneas de transmisión coplanares de impedancia controlada de 50Ω , y vía stitching para evitar resonancias dentro del dieléctrico y confinar los campos electromagnéticos a las líneas de transmisión. También se evitaron, dentro de lo posible, ángulos con transiciones abruptas, se usó ruteo curvo para mantener la integridad de la señal.

Los componentes dispuestos sobre el PCB se modelizan en la figura 5.

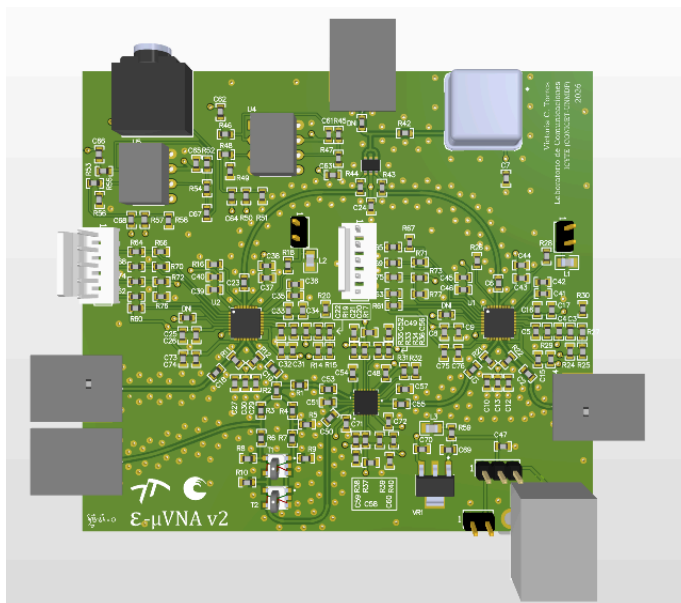


Figura 5: modelo 3D de la implementación en PCB.

2.2. PC

En la figura 6 se puede observar un esquema del hardware que se utiliza para comunicación con el microcontrolador y el ϵ - μ VNA. Se utilizan puertos USB para alimentación de ambas placas. El software se comunica con el microcontrolador por medio de un puerto serie, y recibe las señales del ϵ - μ VNA por medio de la entrada de línea, donde el canal derecho corresponde a la señal medida (proveniente del dispositivo bajo prueba - DUT) y el canal izquierdo tiene la referencia. En general, las entradas de línea poseen una impedancia de entrada de entre 10 y $47k\Omega$ [9] y según mediciones realizadas en el laboratorio, saturan en alrededor de 2Vpp.

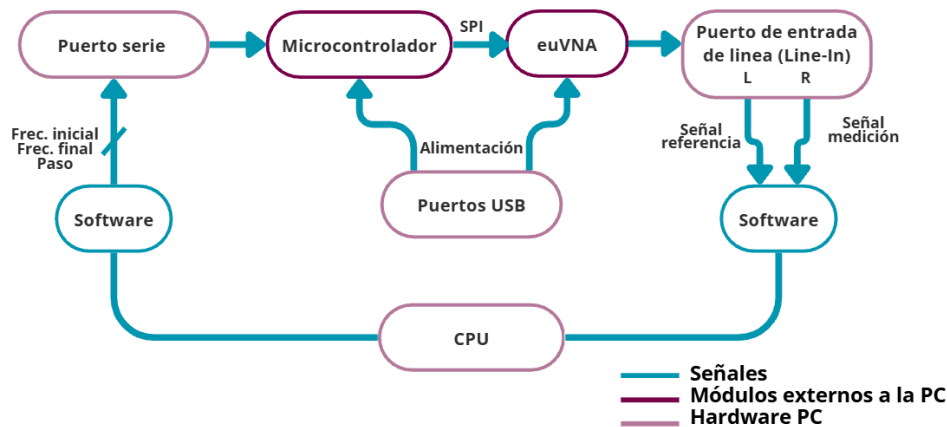


Figura 6: esquema de hardware de PC que se utiliza en el proyecto.

2.3 Microcontrolador

El microcontrolador se implementa utilizando una placa Arduino Uno, que se conecta con el ϵ - μ VNA por medio de cables puente. Se encarga de recibir los datos del software vía puerto serie (frecuencia inicial, frecuencia final y paso) y calcula los registros que luego escribe en los ADF4351 con un protocolo SPI. La transferencia de datos a los registros internos de los ADF es controlada por los pines CLK, DATA, LE y CE.

3. Firmware

El firmware se programa en la plataforma Arduino IDE, en lenguaje C++. El diagrama de flujo del bucle principal es el que muestra la figura 7.

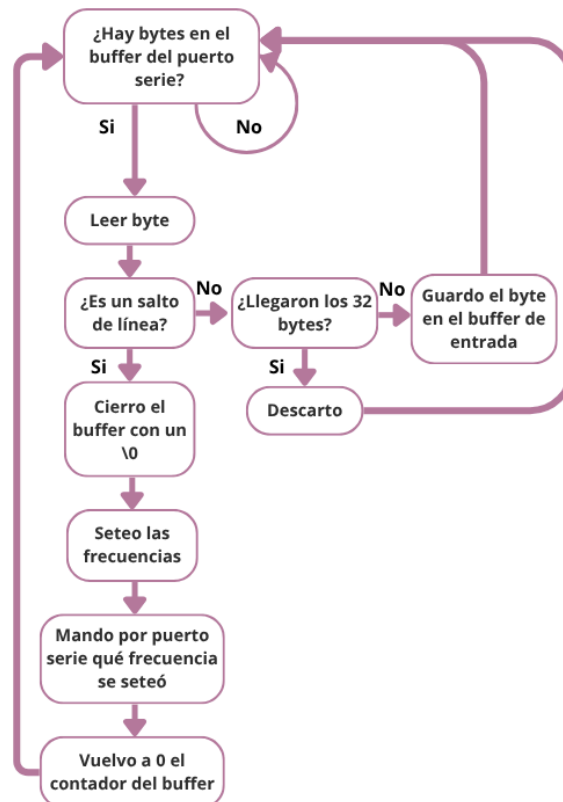


Figura 7: diagrama de flujo del bucle principal del microcontrolador.

En el setup se configuran los pines de salida y la comunicación SPI a 115200 baudios. Se esperan 100 ms y se imprime por puerto serie “Listo”.

Dentro del procesamiento del comando, verifica que la frecuencia inicial sea positiva y ajusta para el ADF2 un offset equivalente a la frecuencia de FI. Luego setea las frecuencias, lo que equivale a calcular los registros como se observa en la figura 8. Por último actualiza los ADF enviando los registros por el pin MOSI del SPI.

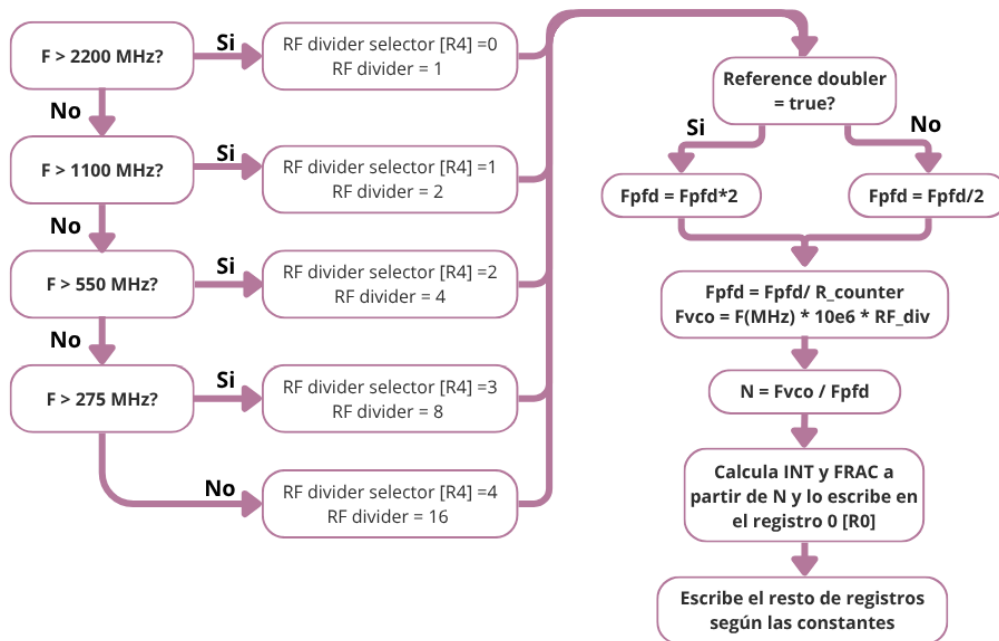


Figura 8: diagrama de flujo del cálculo de registros

4. Software

El software se programa en lenguaje Python, dentro de la aplicación Visual Studio Code. Se puede observar un diagrama de flujos general en la figura 9.

Las bibliotecas a utilizar son, como mínimo: numpy, matplotlib, serial, sounddevice y scipy.signal.

Entre las configuraciones iniciales se encuentran el puerto COM, baud rate, frecuencias de inicio, fin y paso, frecuencia de muestreo de audio, duración de la captura de audio, frecuencia central, ancho de banda y orden del filtro FIR y nombre de los archivos de salida.

El vector de frecuencias se genera con un ciclo for. Luego se diseña el filtro FIR pasabanda con la función “firwin” perteneciente a la biblioteca SciPy. También se establece la conexión serie con la función “serial” de la biblioteca Serial.

Para la adquisición se usa la función “rec” y se obtienen vectores de datos del tipo float64. Se recorta la primera parte para eliminar el transitorio, y se separan los canales en referencia y medición (canal izquierdo y derecho respectivamente).

La FFT compleja de ambos canales se calcula con la función “fft” de la librería NumPy, usando la ventana flattop que permite obtener mejor precisión en amplitud a comparación de otras. Con las funciones “searchsorted” y “argmax” de NumPy se busca el índice del bin de mayor magnitud dentro de la banda del filtro, y luego se calcula S11 en base a los valores obtenidos en la FFT, con el índice mencionado.

Para la calibración se implementa el método SOL de corrección de errores, que consiste en utilizar tres cargas de valores conocidos (Short-Open-Load) para calcular incógnitas asociadas a errores debidos a las adaptaciones de puerto, pérdidas por conectores y errores de directividad. Los valores obtenidos se guardan en un archivo CSV para poder reutilizarlos para mediciones posteriores. En ese caso, se arman nuevamente los vectores de valores complejos leyendo cada línea del archivo.

Por último, se grafican el módulo y fase del parámetro S11 en función de la frecuencia con funciones de la biblioteca Matplotlib.

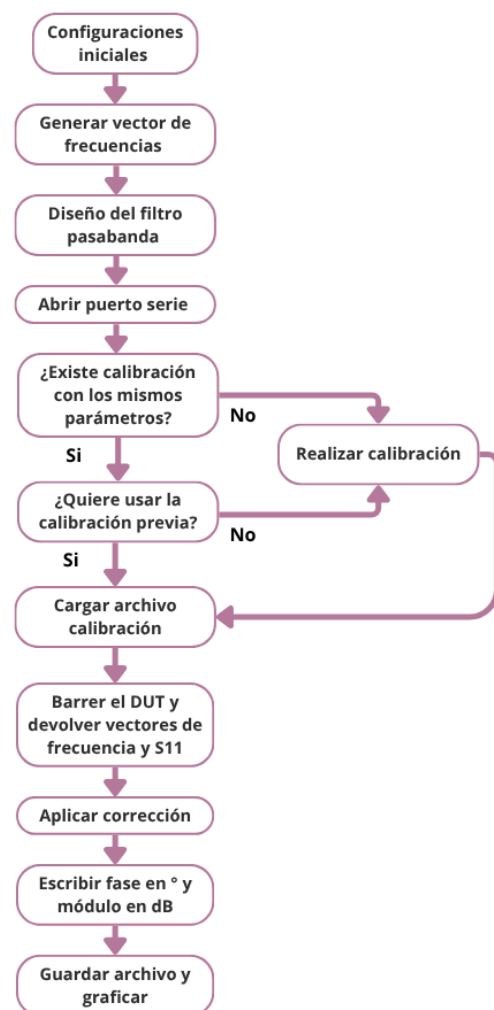


Figura 9: diagrama de flujo general del software

5. Pruebas técnicas

Las pruebas que verifican el correcto funcionamiento del dispositivo son, en principio:

- Medición de tensiones de alimentación.
- Medición de frecuencia del CLK.
- Medición de señal temporal de CLK.
- Prueba con entrada auxiliar de CLK.
- Medición de generación de frecuencia única por parte de ambos generadores ADF.
- Medición de generación de barridos de frecuencias, en ambos generadores ADF.
- Medición de salida de IF a la salida de los amplificadores operacionales.
- Adquisición de audio desde la PC: niveles de señal, FFT.
- Mediciones con cargas conocidas antes y después de calibrar.
- Medición con sensor de glucosa.

6. Referencias

- [1] Cristal oscilador 10MHz, Ecs Inc. Datasheet:
<https://ecsxtal.com/store/pdf/ecs-2200x.pdf>
- [2] Buffer, Diodes Incorporated. Datasheet:
<https://www.diodes.com/assets/Datasheets/74LVC2G07.pdf>
- [3] Sintetizador de RF ADF4351, Analog Devices. Datasheet:
<https://www.analog.com/media/en/technical-documentation/data-sheets/ADF4351.pdf>
- [4] Herramienta de diseño ADISIMPLL 5.9, Analog Devices. descargable en
<https://www.analog.com/en/lp/resources/adisimpll.html>
- [5] Mezclador ADL5802, Analog Devices. Datasheet:
<https://www.analog.com/media/en/technical-documentation/data-sheets/ADL5802.pdf>
- [6] Transformador de RF Mini-Circuits. Datasheet:
<https://www.minicircuits.com/pdfs/TC1-1-13M+.pdf>
- [7] Amplificadores operacionales MCP6023, Microchip Technology. Datasheet:
<https://ww1.microchip.com/downloads/en/DeviceDoc/20001685E.pdf>
- [8] Regulador de tensión lineal UA78M33CDCYR. Texas Instruments. Datasheet:
<https://www.ti.com/lit/ds/slvs059v/slvs059v.pdf?ts=1789190587458>
- [9] <https://www.hispasonic.com/tutoriales/entradas-mic-inst-line-son-para-sirven/43068>

UNMDP-FI

Carrera: Ingeniería Electrónica

***Proyecto:
Desarrollo de un Analizador Vectorial de Redes
para medición no invasiva de glucemia***

Plan de Pruebas



Versión 1.0

1. Alcance

El proyecto propone el diseño, desarrollo, calibración y ensayo de un Analizador Vectorial de Redes (VNA) que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 GHz hasta los 2,2 GHz, adaptado para medición no invasiva de glucemia. Se utilizan circuitos integrados comerciales, dispositivos de montaje superficial (*Surface-Mount Device* - SMD), y conectores. Se aplican técnicas de reflectometría y de diseño en radiofrecuencia.

A continuación se presenta el plan de pruebas para validar el correcto funcionamiento de los módulos que componen el proyecto. Se las puede clasificar en:

- Pruebas unitarias (PU): se concentran en cada componente o módulo del sistema, de forma aislada.
- Pruebas integrales (PI): se realizan a medida que se integran los bloques de componentes.
- Pruebas de homologación (PH): se realizan sobre el sistema completo, validando los requerimientos planteados al inicio del proyecto.
- Pruebas de regresión (PR): se realizan cuando se modifica, ajusta o agrega un módulo o componente.
- Pruebas de aceptación (PA): son las que realiza el usuario final. Se enfocan en características generales y la funcionalidad del sistema.

2. Ambientes de prueba

Todas las pruebas sobre componentes de circuitos integrados se realizarán en un ambiente de laboratorio, con instrumental proporcionado por el Laboratorio de Comunicaciones (LAC) de la Facultad de Ingeniería de la Universidad Nacional de Mar del Plata (UNMDP).

Las pruebas de adquisición se realizan en la aplicación para PC Audacity, las de software en Visual Studio Code.

3. Recursos, herramientas e instrumentos

El instrumental a utilizar comprende los elementos:

- Computadora personal (PC);
- Multímetro Uni-T UT55 [1];
- Osciloscopio Tektronix TDS210 [2];
- Frecuencímetro Optoelectronics Handi-Counter 2810 [3];
- Analizador de espectros Rigol RSA3030N [4];
- Fuente de alimentación regulable Kenwood PA36-3A [5];
- Generador de señales HP 33120A[6];
- Atenuador 110dB HP 8496B [7];
- Cargas SMA estándar;
- Cables y conectores.

4. Políticas de trabajo

Los resultados de las pruebas se reportan a los directores del proyecto: Dr. Ing. Ramiro Avalos Ribas e Ing. Brian Gluzman, quienes participan y supervisan el proceso.

5. Pruebas

5.1 Medición de tensiones de alimentación.

Prueba las conexiones de conectores y cables, planos de alimentación y regulador de tensión.

Tipo de prueba: unitaria.

Resultado esperado: obtener $5\pm 0,2$ V en la entrada del regulador, las alimentaciones de los operacionales y mezclador, y $3,3\pm 0,1$ V a la salida del regulador, en los generadores de RF y oscilador.

Instrumental: Fuente de alimentación, multímetro.

Procedimiento:

- 1) Setear la fuente de alimentación en 5V con una corriente máxima de 0,5A.
- 2) Conectar la fuente de alimentación a los postes de la placa, con el jumper en posición alimentación externa.
- 3) Medir la entrada y salida del regulador de tensión con el multímetro.
- 4) Medir el pin 24, 6 o 13 del mixer.
- 5) Medir el pin 10, 28 o 32 de los generadores de RF.
- 6) Medir el pin 8 (según la hoja de datos) del oscilador.

5.2 Medición de frecuencia y señal temporal del CLK.

Prueba el oscilador a cristal y su integración con el buffer, respectivamente.

Tipo de prueba: unitaria.

Resultado esperado: tener una señal de frecuencia 10MHz, con cruces por cero sin overshoot o ruido apreciable en el osciloscopio.

Instrumental: frecuencímetro y osciloscopio.

Procedimiento:

- 1) Para medir frecuencia, usar el frecuencímetro. Alimentar el oscilador conectando los pines 8 y 1 a 3,3V y 4 a GND. Medir entre output (pin 5) y GND.
- 2) Para medir forma de onda, alimentar el ϵ - μ VNA y medir la salida del buffer (pin 4 o 6).

5.3 Prueba con entrada auxiliar de CLK.

Prueba cómo se comportan los ADF4351 con un cambio en la frecuencia de CLK.

Tipo de prueba: de regresión.

Resultado esperado: con los ajustes pertinentes en el firmware, debería generar correctamente la señal de RF.

Instrumental: PC, generador de señales, analizador de espectros, atenuador.

Procedimiento:

- 1) Soldar una resistencia de 0Ω en "DNI" dejando R42 sin soldar, para alimentar con fuente externa.
- 2) Setear el generador de señales en una señal senoidal o cuadrada de 3,3Vpp, de frecuencia entre 10 y 250 MHz. Tener en cuenta las consideraciones de la hoja de datos del ADF4351 para la frecuencia elegida (por ejemplo una fpfd menor a 125 kHz).
- 3) Escribir los registros para generar una frecuencia arbitraria.
- 4) Conectar el analizador de espectros protegiendo su entrada con el atenuador.
- 5) Medir las salidas auxiliares de los ADF4351. Verificar que genera la frecuencia pedida con la potencia programada.
- 6) Volver a soldar la resistencia R42 dejando DNI sin insertar para usar el oscilador a cristal como entrada de referencia default.

5.4 Medición de generación de frecuencia única por parte de ambos generadores ADF.

Prueba el firmware y la correcta generación de frecuencia por parte de los ADF4351.

Tipo de prueba: unitaria.

Resultado esperado: una señal de la potencia elegida, en la frecuencia elegida.

Instrumental: PC, analizador de espectros, atenuador.

Procedimiento:

- 1) Alimentar el ϵ - μ VNA.
- 2) Programar una frecuencia en el Arduino para que escriba los registros de los ADF.
- 3) Conectar el analizador de espectros protegiendo su entrada con el atenuador, y medir las salidas auxiliares de los ADF. Verificar que genera la frecuencia pedida con la potencia programada, y que los LEDs de enganche se encienden.

5.5 Medición de generación de barridos de frecuencias, en ambos generadores ADF.

Prueba la sección de barrido del software, y la generación de los ADF4351.

Tipo de prueba: unitaria.

Resultado esperado: generación de las frecuencias con la potencia programada en un rango de 1800 a 2200 MHz.

Instrumental: PC, analizador de espectros.

Procedimiento:

- 1) Alimentar el ϵ - μ VNA.
- 2) Conectar el analizador de espectros protegiendo su entrada con el atenuador, y medir las salidas auxiliares de los ADF4351.
- 3) Programar el barrido desde el software, con un ciclo que mande cada frecuencia por puerto serie al Arduino.
- 4) Verificar que genera cada frecuencia pedida con la potencia programada, que los LEDs detectores de enganche se encienden y que el enganche se produce dentro del tiempo programado.

5.7 Medición de salida de IF a la salida de los amplificadores operacionales.

Prueba el módulo FI: mezclador ADL5802 y amplificadores operacionales MCP6023.

Tipo de prueba: unitaria.

Resultado esperado: observar señales de referencia y medición de frecuencia FI.

Instrumental: PC, osciloscopio (opcional).

Procedimiento:

- 1) Alimentar el ϵ - μ VNA.
- 2) Generar una frecuencia única en el ADF principal y la misma con un offset de FI en el ADF que se usa como oscilador local.
- 3) Poner una carga conocida como dispositivo bajo prueba.
- 4) Adquirir la señal con el programa de PC Audacity o medir la salida de los amplificadores operacionales con un osciloscopio. Tener en cuenta que pueden ser señales de muy baja amplitud.
- 5) Observar que se generen las señales incidente/referencia y reflejada/medición, donde la incidente debe tener mayor (o igual) amplitud y se debe tomar de referencia para medir la fase de la reflejada. Si, por ejemplo, el dispositivo bajo prueba es un cortocircuito, la señal reflejada debería tener la fase rotada 180°. Tener en cuenta que puede haber diferencias debido a que no se ha implementado la calibración.

5.8 Adquisición de audio desde la PC: niveles de señal, FFT.

Prueba la adquisición y el cálculo de la FFT a partir de una única frecuencia.

Tipo de prueba: de integración.

Resultado esperado: observar señales de audio de la frecuencia de FI con amplitudes similares a las medidas en la prueba 5.7. Obtener la FFT con un pico máximo en la frecuencia de FI, mínimo 10dB por encima de la tercera armónica.

Instrumental: PC.

Procedimiento:

- 1) Configurar los generadores de RF para obtener una salida en FI.
- 2) Programar la adquisición de audio y el cálculo de la FFT.
- 3) Graficar la onda adquirida en función del tiempo y la FFT.

5.9 Mediciones con cargas conocidas antes y después de calibrar.

Prueba el software integrado con el ϵ - μ VNA.

Tipo de prueba: de homologación.

Resultado esperado: antes de calibrar deberían verse variaciones con respecto a lo esperado para las cargas estándar. Luego de calibrar se debería observar correctamente o con mínimas variaciones el coeficiente de reflexión esperado

Instrumental: PC.

Procedimiento antes de calibrar:

- 1) Programar el Arduino para que espere las frecuencias por puerto serie y calcule y escriba los registros.
- 2) Colocar como dispositivo bajo prueba una carga de valor conocido (cortocircuito, circuito abierto, 50 Ω)
- 3) Ejecutar el barrido y calcular el S11 para todas las frecuencias.
- 4) Mostrar en un gráfico de módulo [dB] y fase [°] el S11 calculado.

Procedimiento luego de calibrar:

- 1) Elegir un barrido y realizar la calibración para ese barrido.
- 2) Colocar una carga de coeficiente de reflexión conocido
- 3) Correr el programa y evaluar resultados.

5.10 Medición con sensor de glucosa.

Prueba el sistema completo en su capacidad para detectar una resonancia entre 1800 y 2200 MHz.

Tipo de prueba: de aceptación.

Resultado esperado: debería observarse una curva de módulo cercano a 0 y luego un pico de resonancia alrededor de los 2000 MHz.

Instrumental: PC.

Procedimiento:

- 1) Realizar la calibración para un barrido de entre 1800 y 2200MHz, con un paso de 1MHz o menor.
- 2) Conectar el sensor de glucemia como dispositivo bajo prueba.
- 3) Correr el programa y evaluar resultados.

6. Referencias

- [1] Manual Tester UT55. Disponible en:
https://www.tme.com/Document/9d0acda5b1c310707bd15e5f784f9bae/ut55_en.pdf
- [2] Manual osciloscopio Series TDS200. Disponible en:
<https://www.tek.com/en/oscilloscope/tds210-manual/tds200-series-user-manual>
- [3] Manual frecuencímetro. Disponible en:
https://www.optoelectronics.com/_files/ugd/c0ec7f_ab98242f8c3842b889cf7a8ff2227f43.pdf
- [4] Manual analizador de espectros Rigol Series RSA3000. Disponible en:
<https://telonic.co.uk/jg/wp-content/uploads/2021/05/Rigol-RSA3000-UserGuide.pdf>
- [5] Datasheet Kenwood PA36-3A. Disponible en: <https://www.ntecusa.com/docs/pa-a.pdf>
- [6] Datasheet HP 33120A. Disponible en:
<https://www.keysight.com/us/en/assets/7018-06809/data-sheets/5968-0125.pdf>
- [7] Datasheet atenuador HP8496B. Disponible en:
<https://www.keysight.com/us/en/assets/9018-01300/user-manuals/9018-01300.pdf?success=true>

Bibliografía

- [1] Sitio Web Oficial. URL: <https://www.freestylelibre.com.ar/>.
- [2] Altium. *Consideraciones para el diseño de PCB de alta velocidad: Prácticas recomendadas para la colocación y el enrutamiento de condensadores de desacoplo*. Dec. 2020. URL: <https://resources.altium.com/es/p/high-speed-pcb-design-considerations-bypass-capacitor-place-and-route-tips>.
- [3] Altium. *Via Stitching and Via Shielding in PCB Design*. s. f. URL: <https://www.altium.com/documentation/altium-designer/via-stitching-via-shielding-pcb>.
- [4] Altium Limited. *Altium Designer*. Version 26.1.1. Licencia educativa. 2026. URL: <https://www.altium.com>.
- [5] Analog Devices. *HMC601 75 dB logarithmic detector/controller*. <https://www.analog.com/en/products/hmc601.html>. n.d.
- [6] Tektronix Inc. *Vector Network Analysis: A Practical Introduction*. 2023. URL: https://download.tek.com/document/70W_60918_0_Tek_VNA_PR1.pdf?_gl=1*wh717q*_gcl_au*MjQyMjEyMTE5LjE3ODk0ODkzMjU.*_ga*NjkwMjAxNzcxLjE3ODk0ODkzMjU.*_ga_1HMYs1JH9M*cze30TAxODY4MjkkbzIkZzAkDDE30TAxODY4MzIkajU3JGwwJGgwJGRFXzVmVFRzOGMwbmFqaG8zX
- [7] Keysight Technologies. *778D Coaxial Dual-Directional Coupler, 100 MHz to 2 GHz*. s. f. URL: <https://www.keysight.com/us/en/product/778D/coaxial-dualdirectional-coupler-100-mhz-to-2-ghz.html>.
- [8] Dimitris G. Manolakis and Vinay K. Ingle. *Applied Digital Signal Processing: Theory and Practice*. Cambridge, UK: Cambridge University Press, 2011. ISBN: 978-0-521-11002-0.
- [9] Mini-Circuits. *ZABDC20-322H-S+ Coaxial Directional Coupler*. s. f. URL: <https://www.minicircuits.com/WebStore/dashboard.html?model=ZABDC20-322H-S%2B>.
- [10] World Health Organization. *Diabetes*. Sitio Web Oficial. Nov. 2024. URL: <https://www.who.int/news-room/fact-sheets/detail/diabetes>.
- [11] World Health Organization. *Global report on diabetes*. Sitio Web Oficial. 2016. URL: <https://www.who.int/publications/i/item/9789241565257>.
- [12] Hewlett Packard Inc. Rytting D. “Network analyzer error models and calibration methods”. In: *HP* (1998).
- [13] Urvashi Sengal. *Directional Couplers: Their Operation and Application*. Mini-Circuits. Oct. 2020. URL: <https://blog.minicircuits.com/directional-couplers-their-operation-and-application/>.
- [14] E. Topsakal, T. Karacolak, and E. C. Moreland. “Glucose-dependent dielectric properties of blood plasma.” In: *Proceedings of the XXXth URSI General Assembly and Scientific Symposium* (2011). DOI: 10.1109/URSIGASS.2011.6051324.